(Integration Testing and) Scenario-Based Tests for Cyber-Physical Systems Autonomous Driving Systems

Alexander Pretschner, TUM, fortiss, and bidt

Marktoberdorf Summer School 2026

August 12th-August 15th, 2026

jww Michael Wolf, Florian Hauer, Nicola Kolb, Tabea Schmidt, Thomas Hutzelmann

Storyline

Integration Tests

Generate good test cases from existing catalog of scenario types

- No regression tests for closed-loop systems!
- Optimization problem: describe scenario types and objective functions
- Fitness functions
- Revisiting Re-Use
- STL
- Completeness

Derive parameterized scenarios from data

- Clustering
- Clusters and their semantics
- Completeness revisited: recorded drives

Discussion and Conclusions

Framing the Content

Testing, not formally verifying

Integration testing and system-level testing, not component-level

Continuous/hybrid systems, not discrete

No specification: testing the untestable

Testing

Test case: input, expected output, environment conditions

Testing: apply input to system under test, compare actual to expected output

Good test case: detect potential defect with good cost effectiveness (reduce risk!)

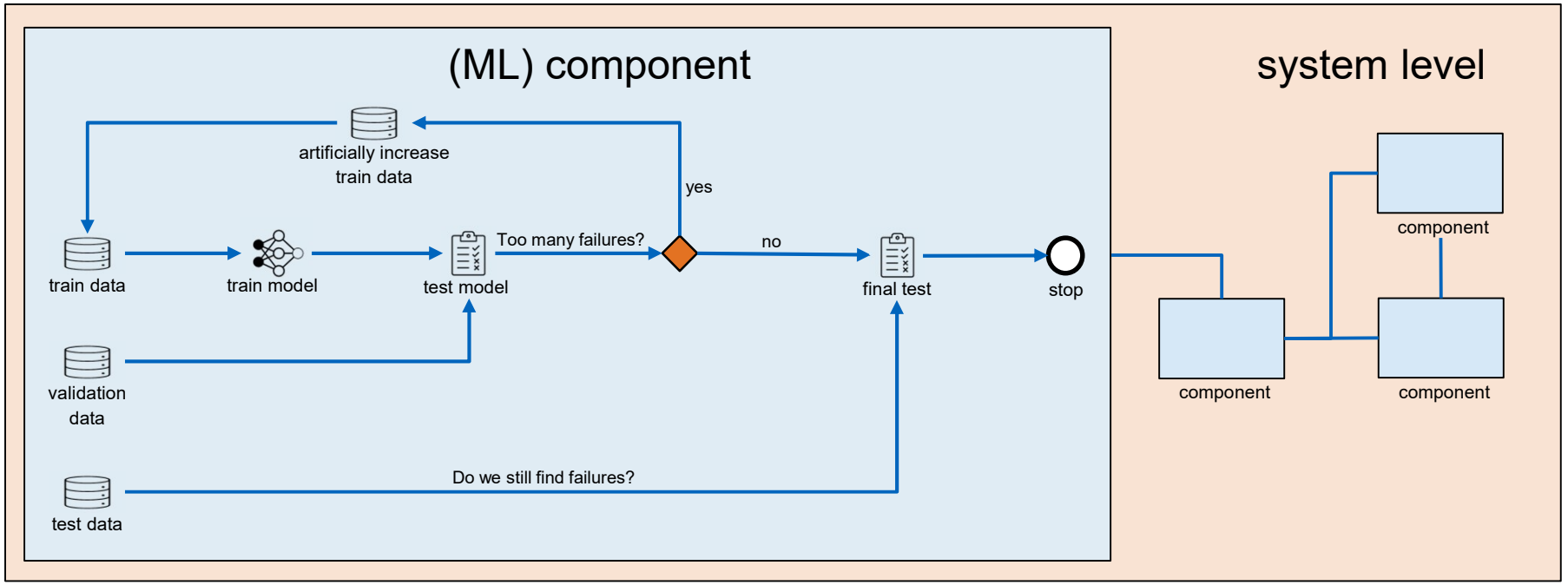
„Testing can never show the absence of defects …“ Yet

- Scalable;
- Practically applicable to entire systems rather than single components, including hardware;
- Depending on test level, done under more realistic circumstances with fewer strong assumptions

System-Level not Component-Level

Software runs on complex stacks: libraries, virtual machine, middleware, operating system, hardware; connected SW/HW systems; sensors and actuators

Many of these parts come without source code



Test Levels

Unit

Integration

System

Acceptance

What do we test when and how?

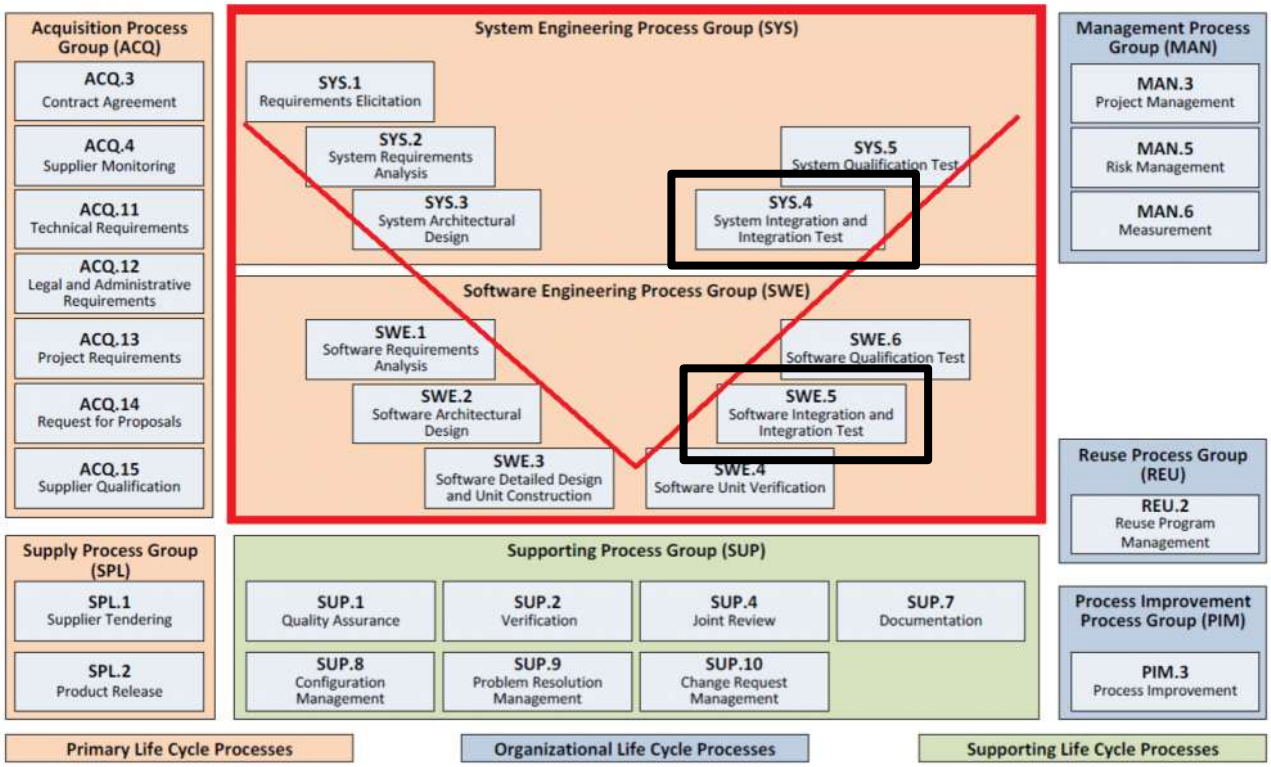
Technical software functions: unit & subsystem tests, horizontal software integration tests
- discrete rather than continuous behavior
- classical unit testing frameworks

User functions: SiL tests
- (discrete behavior: maybe unit testing frameworks)
- specifically continuous behavior: SiL, cosimulation with FMUs

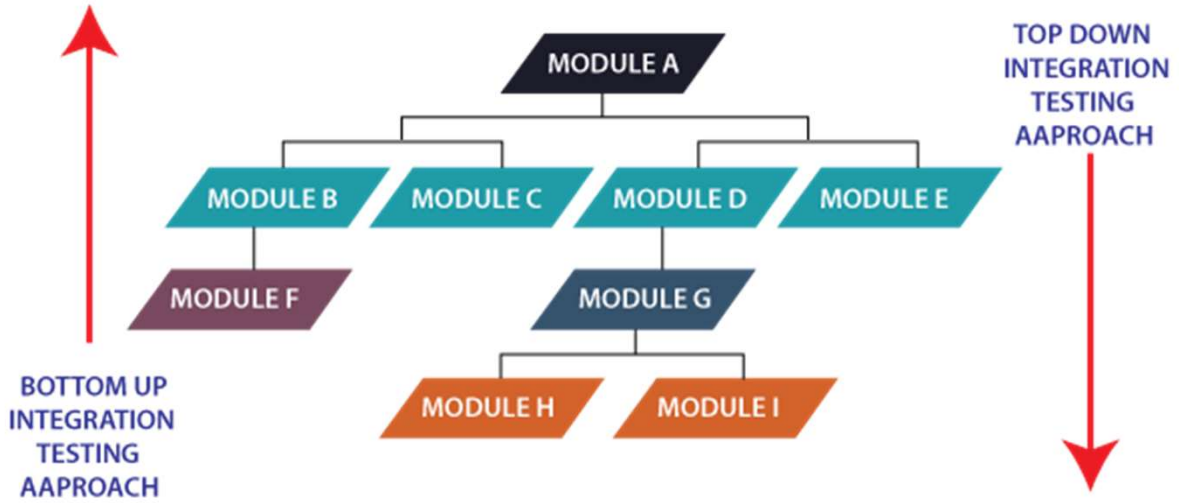
Systemic properties (later): SiL or HiL tests

Let‘s start with integration tests.

ASPICE and Integration Testing



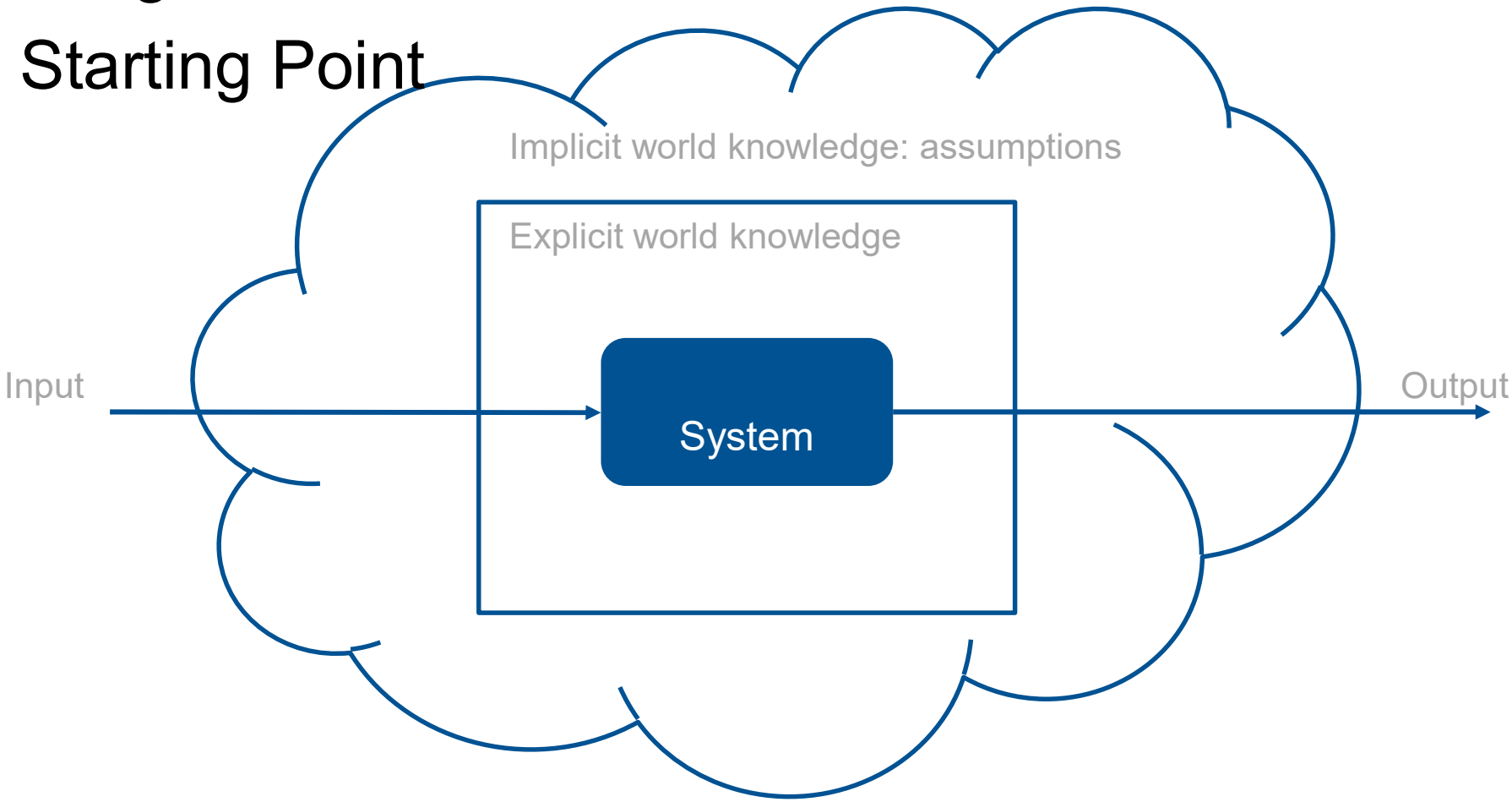
Classical Software Integration Testing



https://www.javatpoint.com/top-down-vs-bottom-up-integration-testing

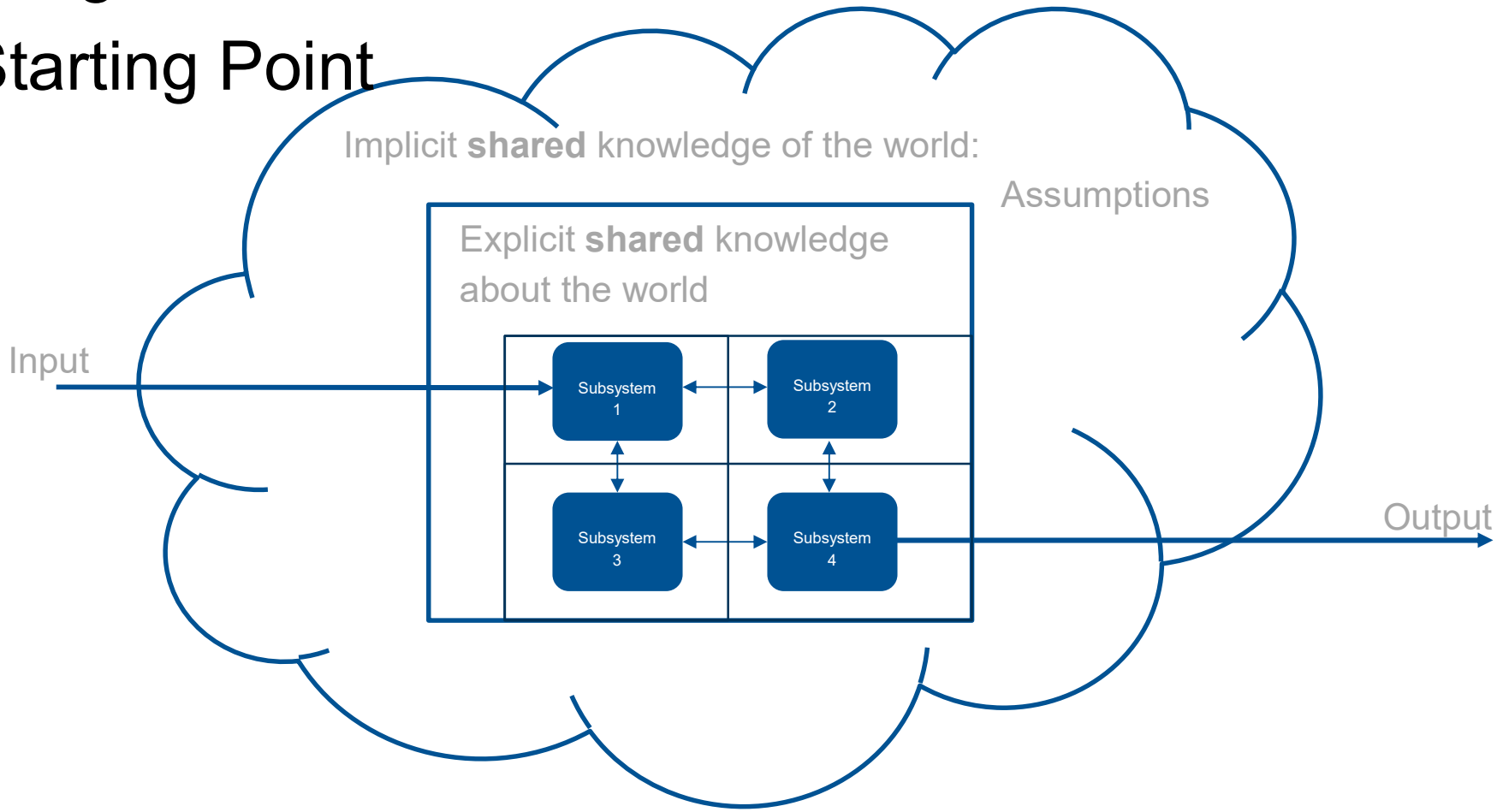
Integration Tests

Starting Point



Integration Tests

Starting Point



Why Integration is a Challenge

Different understanding/assumptions for different stakeholders

Wrong understanding/assumptions

Incomplete understanding/assumptions

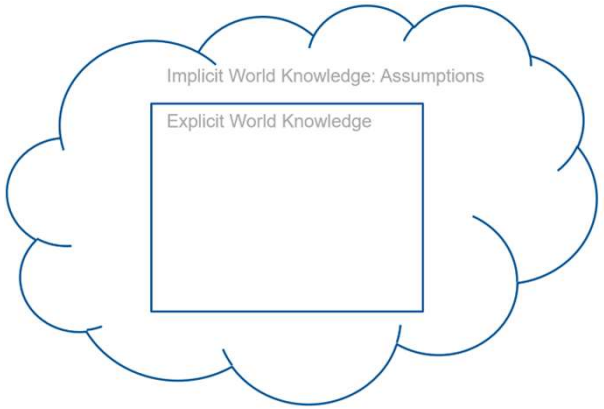
Sometimes realistically foreseeable:

Some system properties

Sometimes not:

Externalities, feature interactions

some (emergent) system properties



Dimensions

„Units“: classes, tasks, things, (micro) services, ECUs, …

Development process: V, agile, distributed

Test during development, test by system integrator

System class: CPS, focus on software, distributed

Integration Faults as Design Faults in Disguise

Integration testing usually is a specification problem.

1. **Integration is based on a shared understanding of the world**

   Explicit contracts (APIs, data models) plus implicit assumptions: timing, units, ranges, ordering.

2. **Integration faults surface only under composition**

   Every part meets its own spec, yet the system fails — the fault was in the design or infrastructure.

3. **In principle, the unit / integration boundary is artificial**

   With sufficiently complete specs we'd catch them earlier. The real gap is the missing specification.

4. **Remedy 1: Test against negative specifications**

   No spec? Catalog the specific faults composition can produce — the systemic ones it can't reach.

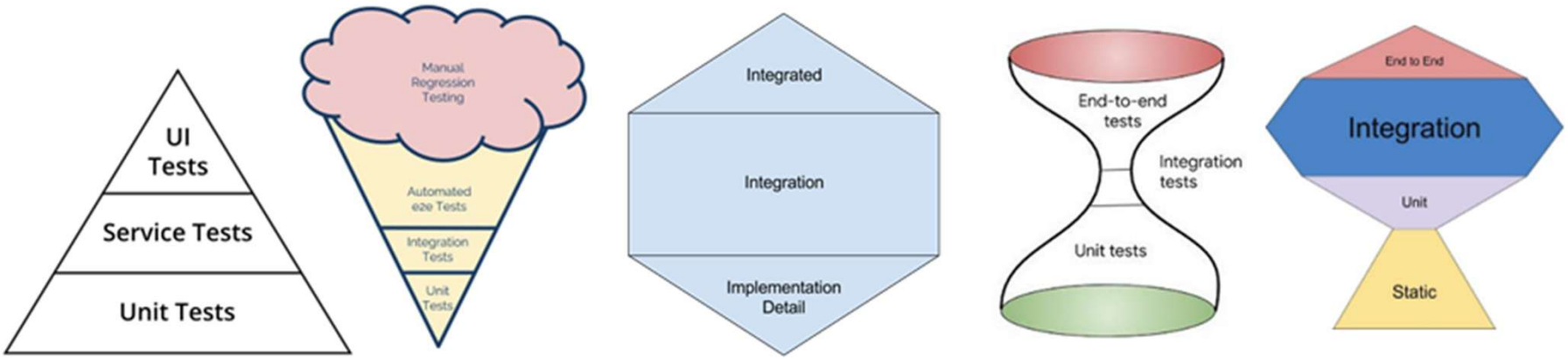
5. **Remedy 2: Surface the implicit background knowledge**

   Generative AI and compound engineering can help fill in the unwritten assumptions.

Quotations from Practitioners

- „We often think, „**this bug could have been detected earlier“ – that our tests are redundant**“
- „**Integration tests test if we have had the same understanding**“
- „Integration tests are tests where a data base is involved“
- „Unit tests have no state, integration tests do“
- „Units can‘t be tested in a SIL feedback loop“
- „**Testing time-dependent behavior doesn‘t make sense for units**“
- „Unit tests are technical, integration tests test user functionality“
- „Regression tests can only be used for units“
- „We can‘t really distinguish between integration and (sub)system tests“
- „**The real problem with integration are sporadic failures**“
- „Different integration stages lead to different definitions“
- „Compatibility biggest problem when there are many versions&variants“

Test „Pyramids“ and Shift-Left

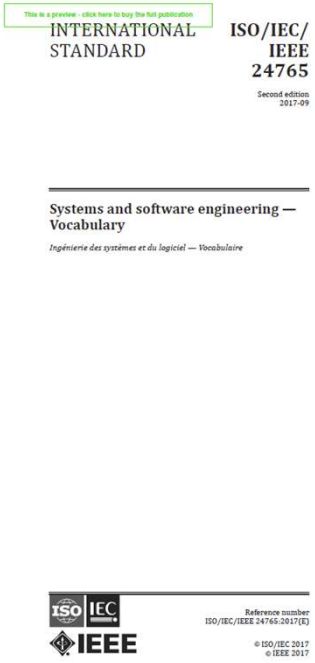


Confusion

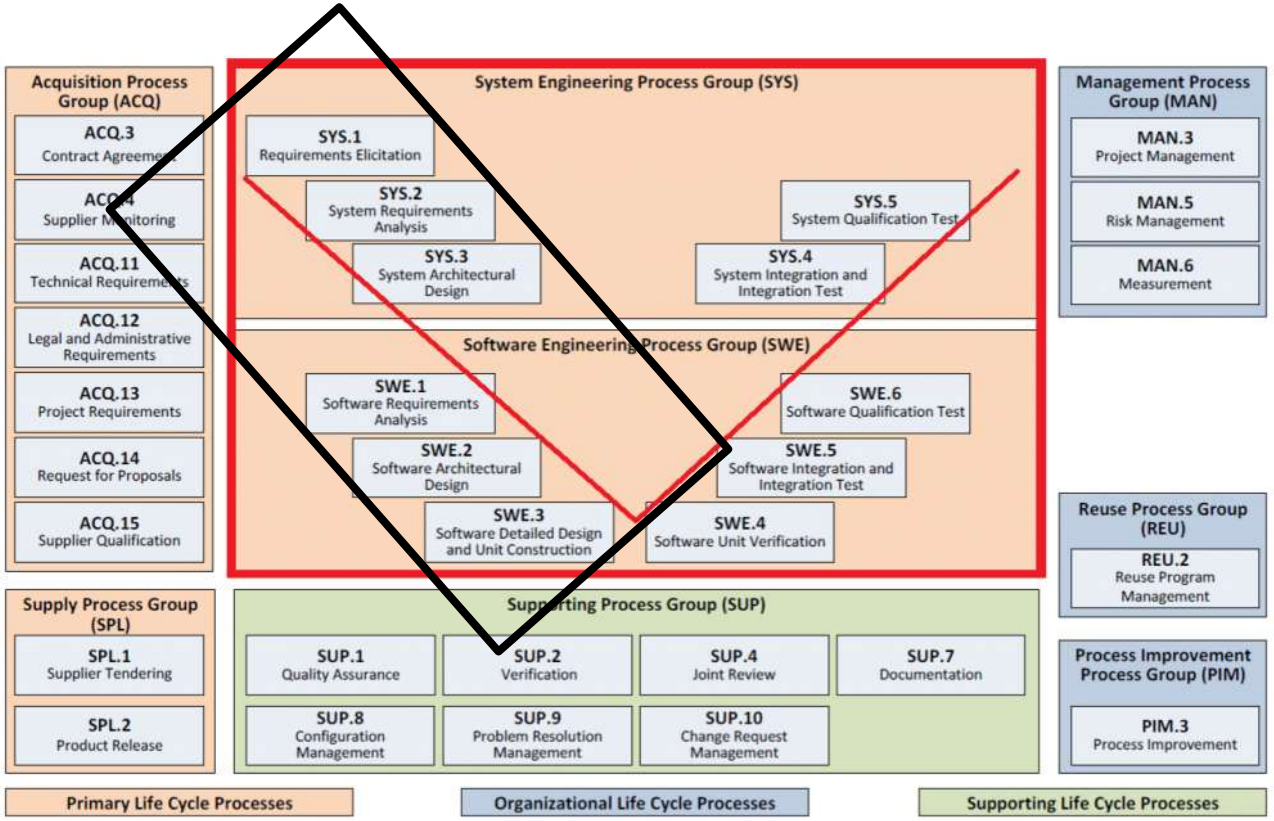
Integration Test [ISO/IEC/IEEE 24765:2017(E)]:
"progressive **linking and testing of programs or modules** in order to ensure their **proper functioning** in the complete system"

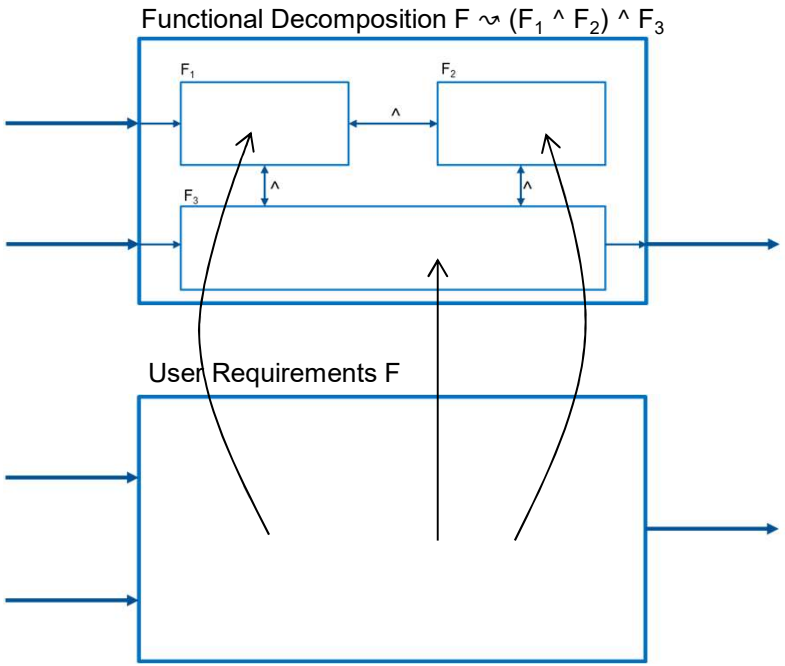
Integration Testing [ISO/IEC/IEEE 24765:2017(E)]:
"**testing** in which software **components**, hardware components, or both are **combined** and tested to **evaluate the interaction among them**"

ASPICE: doesn‘t define!
SWE.5 to verify components against the detailed design, integrate the software elements,
and **verify that the integrated elements comply with the software architecture**, **including the interfaces between them**.
SYS.4 analogous at system level: integrating system items and verifying the integrated system against the system architecture

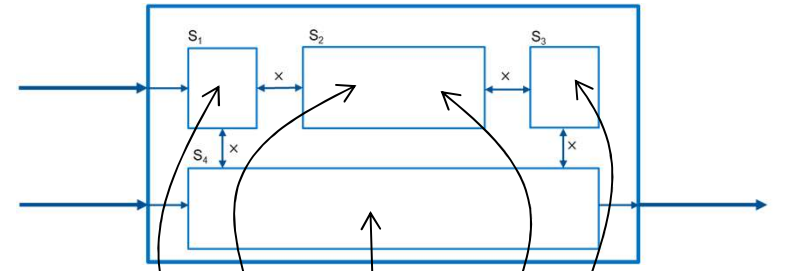


… Let's Start with the Left Branch of the V

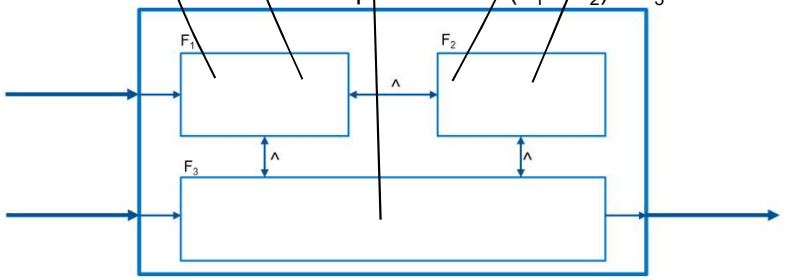



Functional Decomposition F ~ (F1 ^ F2) ^ F3
F1
F2
^
^
^
F3
User Requirements F

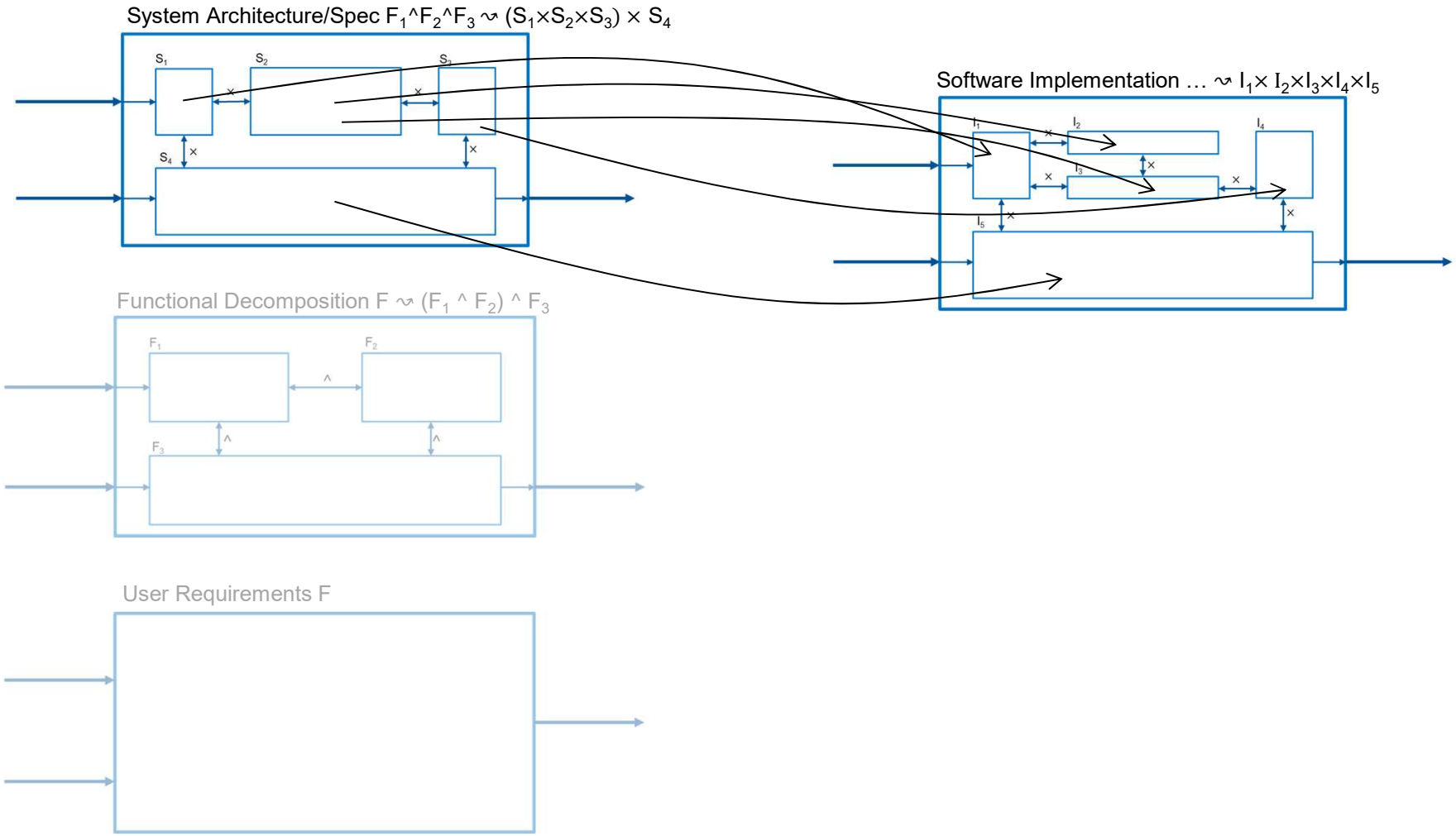
System Architecture/Spec $F_1 \hat{} F_2 \hat{} F_3 \rightsquigarrow (S_1 \times S_2 \times S_3) \times S_4$

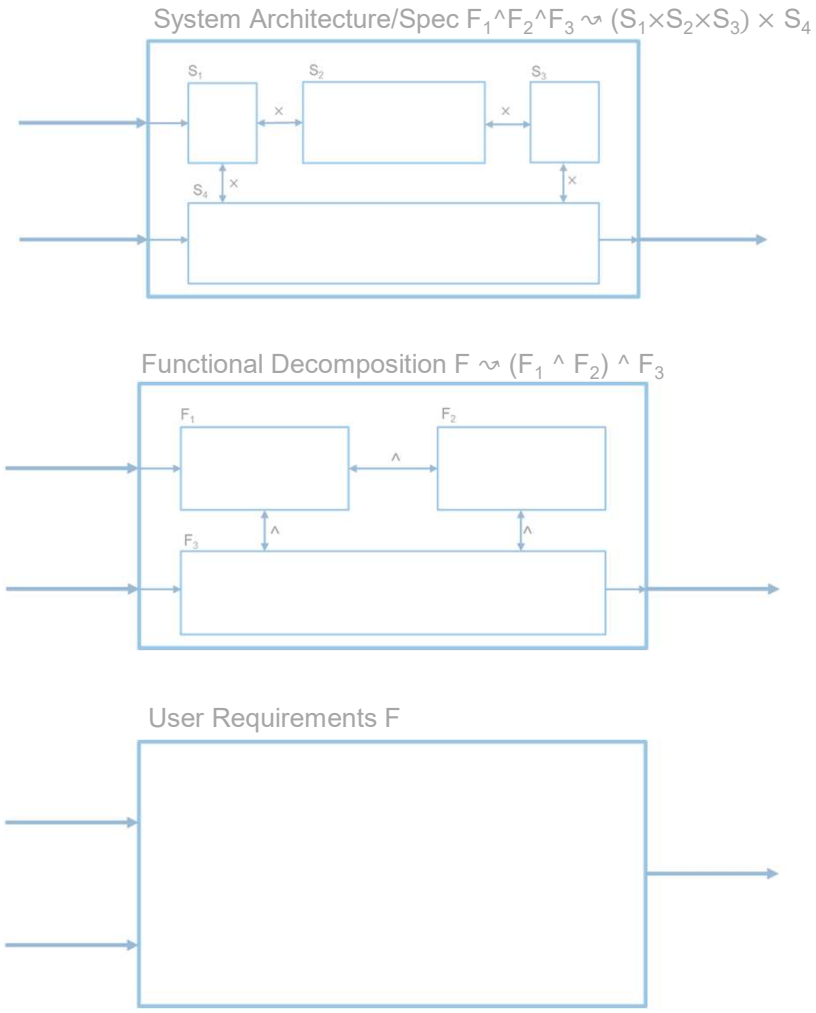


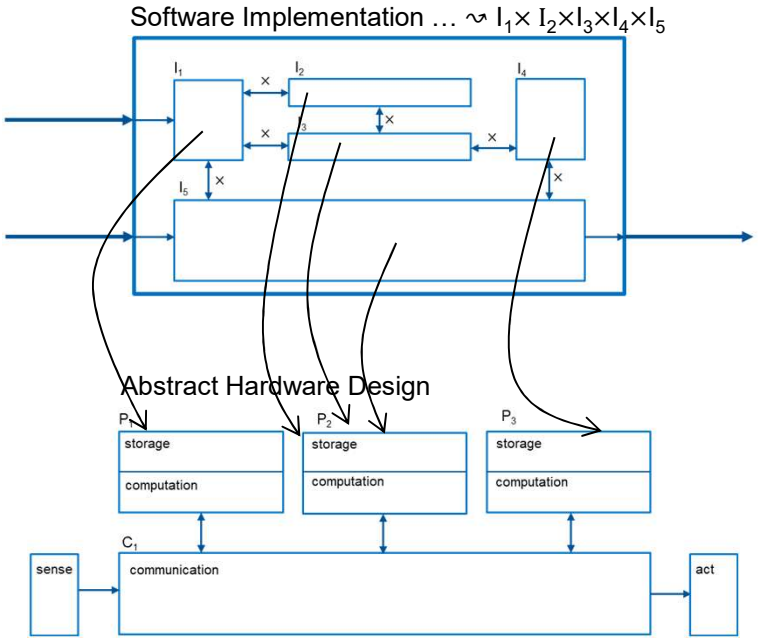
Functional Decomposition $F \rightsquigarrow (F_1 \hat{} F_2) \hat{} F_3$

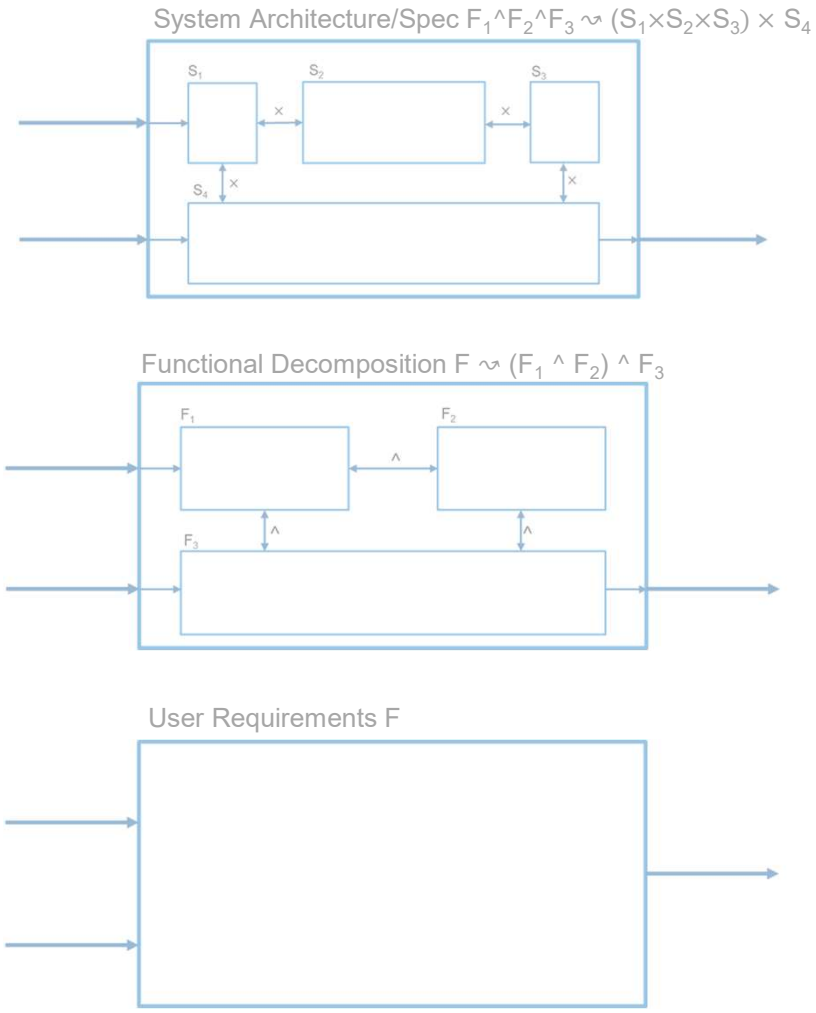


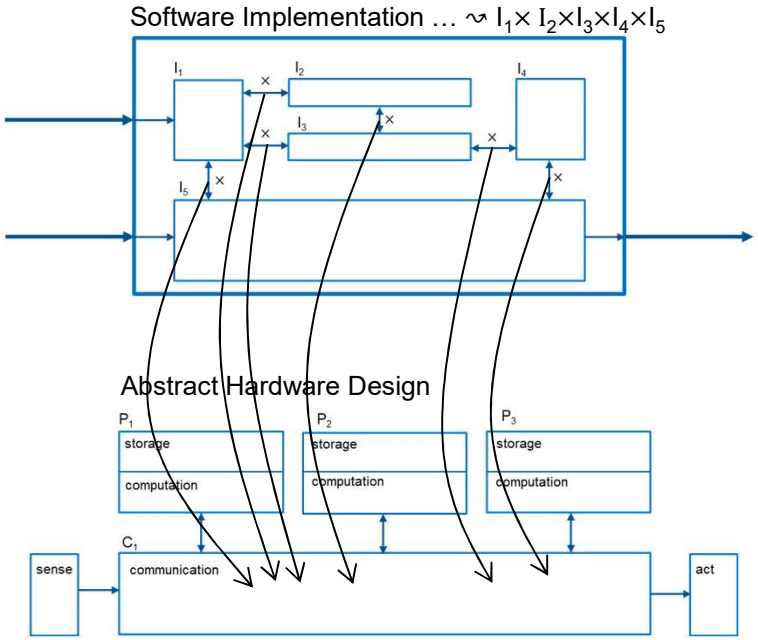
User Requirements F

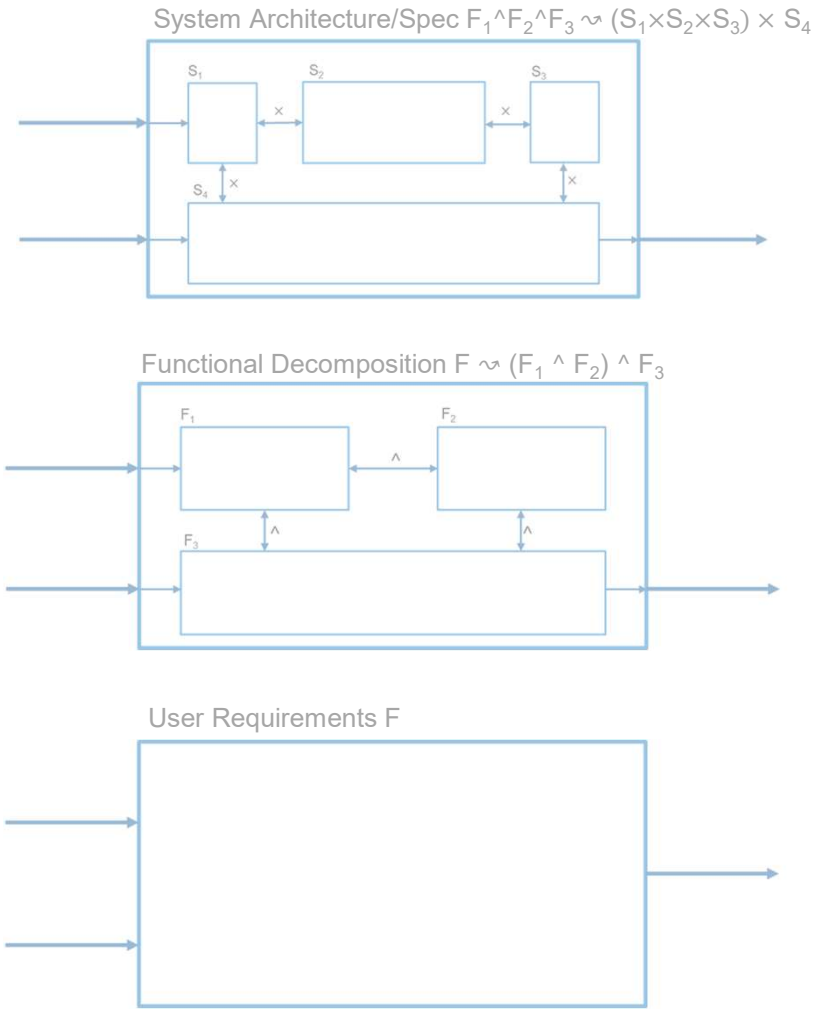

System Architecture/Spec F1^F2^F3 ⇝ (S1×S2×S3) × S4
S1
S2
S3
S4
Software Implementation … ⇝ I1× I2×I3×I4×I5
I1
I2
I3
I4
I5
Functional Decomposition F ⇝ (F1 ^ F2) ^ F3
F1
F2
F3
User Requirements F

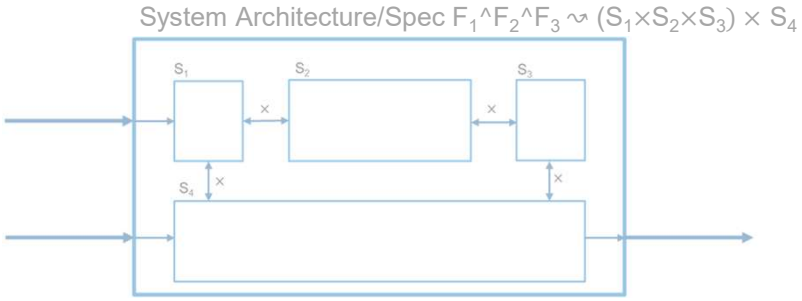

System Architecture/Spec F_1^F_2^F_3 ⇝ (S_1×S_2×S_3) × S_4
S_1
S_2
S_3
S_4
Functional Decomposition F ⇝ (F_1 ^ F_2) ^ F_3
F_1
F_2
F_3
User Requirements F

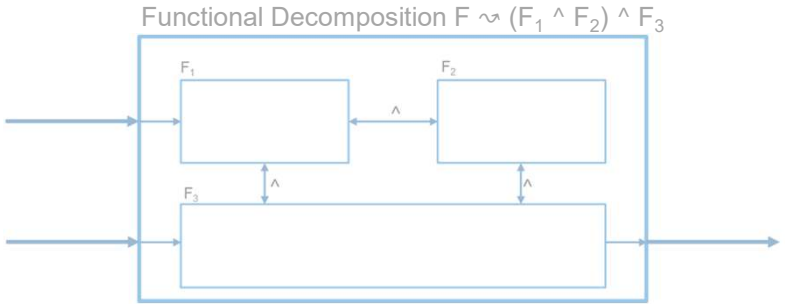

Software Implementation … ⇝ I_1× I_2×I_3×I_4×I_5
I_1
I_2
I_3
I_4
I_5
Abstract Hardware Design
P_1
P_2
P_3
storage
computation
C_1
communication
sense
act


System Architecture/Spec F_1^F_2^F_3 ⇝ (S_1×S_2×S_3) × S_4
S_1
S_2
S_3
S_4
Functional Decomposition F ⇝ (F_1 ^ F_2) ^ F_3
F_1
F_2
F_3
User Requirements F

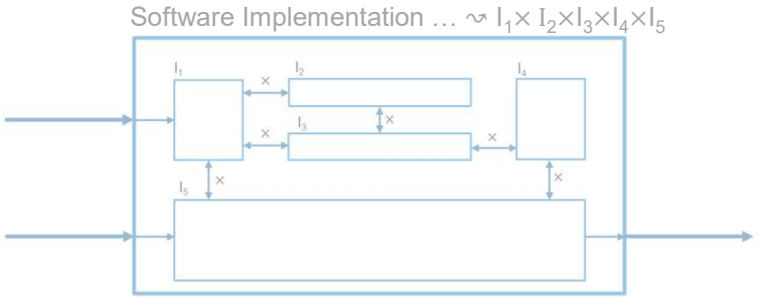

Software Implementation … ⇝ I_1× I_2×I_3×I_4×I_5
I_1
I_2
I_3
I_4
I_5
Abstract Hardware Design
P_1
P_2
P_3
storage
computation
C_1
communication
sense
act

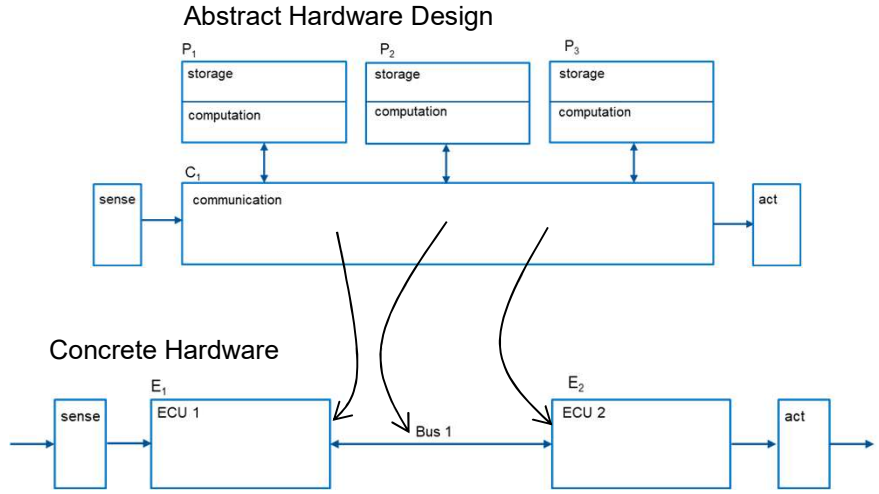

System Architecture/Spec F_1^F_2^F_3 ⇝ (S_1×S_2×S_3) × S_4
S_1
S_2
S_3
S_4
Functional Decomposition F ⇝ (F_1 ^ F_2) ^ F_3
F_1
F_2
F_3
User Requirements F
Software Implementation … ⇝ I_1× I_2×I_3×I_4×I_5
I_1
I_2
I_3
I_4
I_5
Abstract Hardware Design
P_1
P_2
P_3
storage
computation
C_1
communication
sense
act
Concrete Hardware
E_1
E_2
ECU 1
ECU 2
Bus 1


System Architecture/Spec F_1^F_2^F_3 ⇝ (S_1×S_2×S_3) × S_4
S_1
S_2
S_3
S_4


Functional Decomposition F ⇝ (F_1 ^ F_2) ^ F_3
F_1
F_2
F_3


User Requirements F


Software Implementation … ⇝ I_1× I_2×I_3×I_4×I_5
I_1
I_2
I_3
I_4
I_5


Abstract Hardware Design
P_1
P_2
P_3
storage
computation
C_1
sense
communication
act
Concrete Hardware
E_1
E_2
ECU 1
ECU 2
Bus 1

Integration Levels

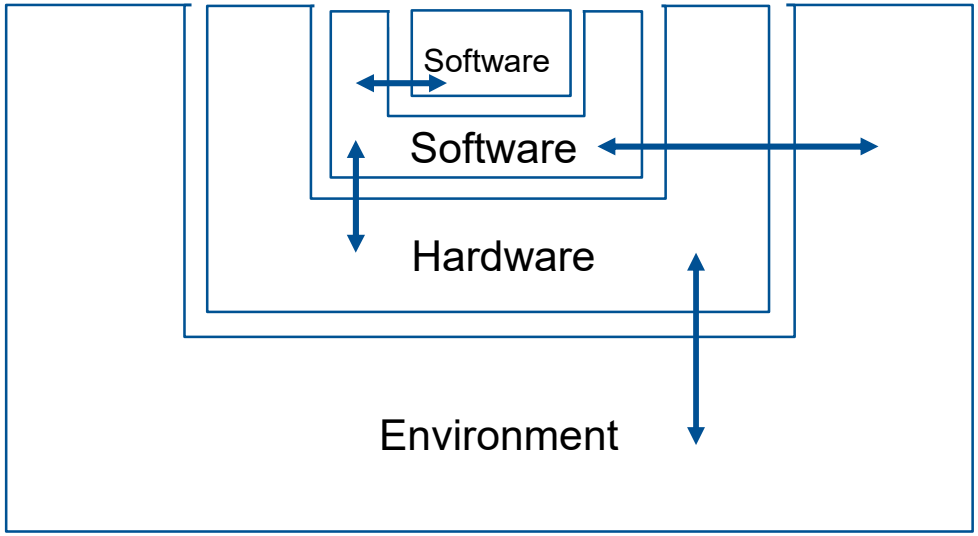
Unit

Horizontal SW integration

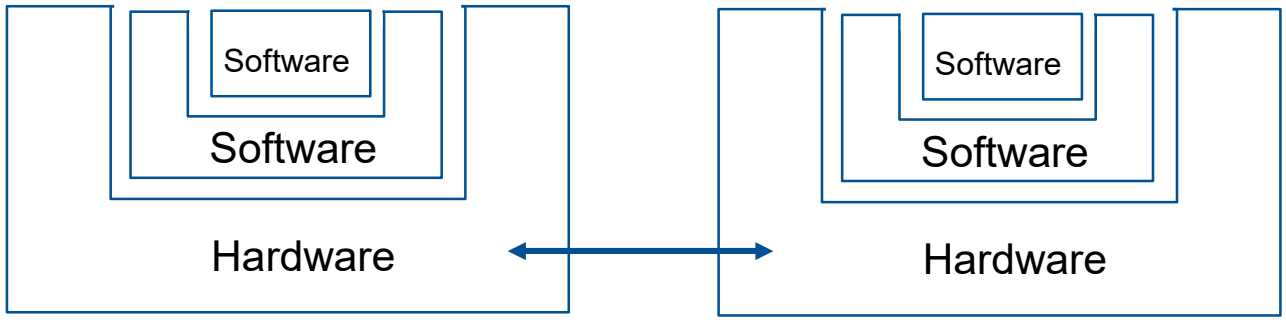
Vertical SW/HW integration

SiL

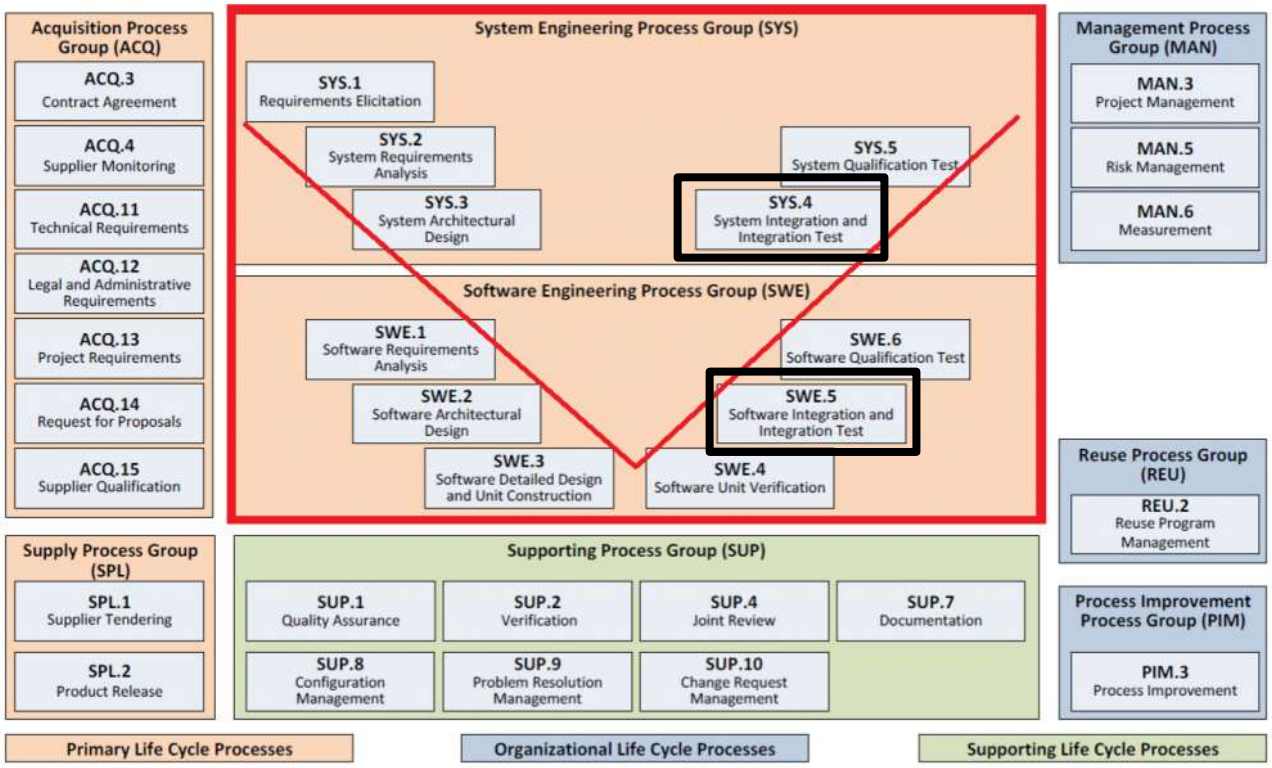
HiL



… and Horizontal HW Integration



… That's the Right Branch of the V



In Principle: Design Steps are Refinements

Requirements become functional system requirements $F_1..F_k$
System requirements become subsystem specifications $S_1..S_l$
Subsystem specs become component architectures $J_1..J_m$
Component architectures become software implementations $I_1..I_n$
Software implementations become deployments $D_1..D_o$

$$R \rightsquigarrow F_1 + \cdots + F_k \rightsquigarrow S_1 + \cdots + S_\ell \rightsquigarrow J_1 + \cdots + J_m \rightsquigarrow I_1 + \cdots + I_n \rightsquigarrow D_1 + \cdots + D_o$$

$+$ means different things: function composition, logical conjunction, function calls, message protocols

Descriptions at each level necessarily partial; each next level removes some partiality *and adds assumptions* about the context of any F,S,J,I,D (implicit world knowledge!)

And yes, the world is not a waterfall.

Why Integration Tests?

Development is $R \rightsquigarrow F_1 + \cdots + F_k \rightsquigarrow S_1 + \cdots + S_\ell \rightsquigarrow J_1 + \cdots + J_m \rightsquigarrow I_1 + \cdots + I_n \rightsquigarrow D_1 + \cdots + D_o$

If refinement process was linear and correct-by-design, no need to integration test: we'd have $D_1 + \cdots + D_o \Rightarrow R$!

Because of the assumptions about the context of any F,S,J,I,D, usually not the case.

Hence *need to check*! Directly checking „deployed system D satisfies requirements R", $D \Rightarrow R$, too difficult, and so is $D \Rightarrow I$, $D \Rightarrow J$, $D \Rightarrow S$, $D \Rightarrow F$.

This problem solved by right branch of the V. We first unit test $D_i \Rightarrow I_j$.

Integration testing then means to identify **subsets of components** D' or I' **and their respective specifications** S', J', F' and check if $D' \Rightarrow S'$, $D' \Rightarrow J'$, $D' \Rightarrow F'$ or $I' \Rightarrow S'$, $I' \Rightarrow J'$, $I' \Rightarrow F'$

What‘s the Problem?

How can it be that $[I_1] \rightarrow [J_1]$ and $[I_2] \rightarrow [J_2]$ but not $[I_1 + I_2] \rightarrow [J_1 + J_2]$?

Because we didn‘t really show $[I_i] \rightarrow [J_i]$!

- Testing samples $\uparrow$ rather than proves $\rightarrow$
- Tests implicitly encode additional assumptions A_i for some deployment context D
- Specs make (implicit) assumptions W about runtime environment W_real: communication infrastructure, hardware

In reality, we approximate $[I_i] \rightarrow [J_i]$ at three levels: $[I_i + A_i + W\_real] \uparrow [J_i + W]$

(Formal verification proves and makes assumptions explicit.
Yet these may still be wrong or partial: Specs are abstractions!)

Assumptions

Between components:

- the payload is JSON
- timestamps are UTC and strictly increasing
- each order-ID reaches me at most once
- forces are in newton-seconds
- the input list is shorter than 1024 entries
- I am never called concurrently

Across components, infrastructure:

- No overflows
- arbitrary precision
- little-endian-encoding
- synchronous communication
- lossless FIFO communication, latency <10ms
- exactly-least-once-communication

***In testing, assumptions are almost always implicit**: **each harness constructs its own unit's environment, so A_i hold automatically** and nobody has a reason to write them down.*

In a proof, A_i must be stated to get the proof through. They may or may not hold.

Hyrum's law: What matters is the observable behavior, not the contract.

Obligations for $[I_1+A_1+I_2+A_2+W_{real}] \uparrow [J_1+J_2+W]$

(i)	$[D] \rightarrow [A_1]$ & $[A_2]$	The deployment context must establish the assumptions
(ii)	[W_real] → [W]	The platform must satisfy its own description
(iii)	$[I_1 + A_1] \uparrow [J_1]$ $[I_2 + A_2] \uparrow [J_2]$	Each implementation must satisfy its own specification
(iv)	[X + Y] = [X] & [Y]	+ must be compositional

These four are the four fault classes.
An integration fault is the breach of at least one of them.

Sources of Integration Faults

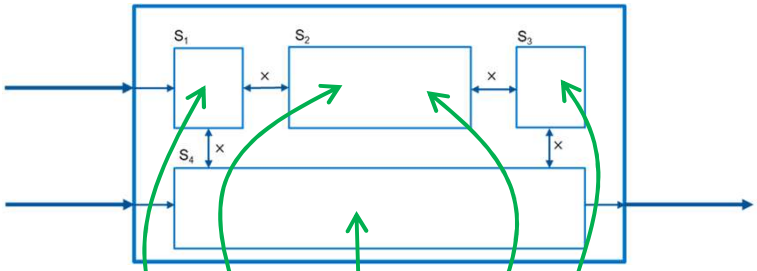
Class	Obligation	Category	Attribution	Repair
(a)	(i) $[A_1]$ & $[A_2]$ established	**Design**	none — three admissible sites	interfaces, adapter, protocol, decomposition
(b)	(ii) [W_real] → [W]	**Infrastructure**	none — beneath all units	platform; or correct I and redesign
(c)	(iii) $[I_1+A_1]\rightarrow[J_1]$, $[I_2+A_2]\rightarrow[J_2]$	Unit fault	one component or its environment	code or spec of that component
(d)	(iv) Compositionality	**Design, meta**	impossible — fault in the frame	choice of formalism; resource architecture

Why design / infrastructure is not an arbitrary split

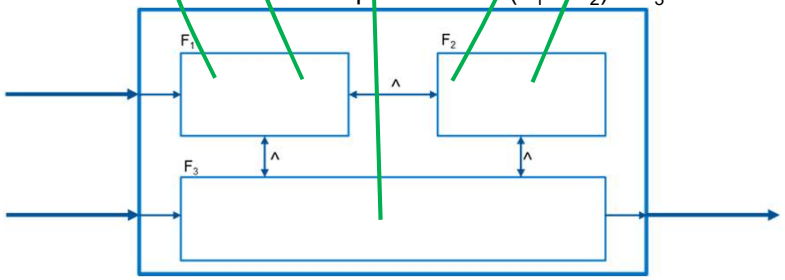
Integration faults cannot be canonically attributed to one component.
A fault attributable to no component has two possible sources:
the relation between the units, which the design step fixed or the layer beneath them.

(c) denotes an integration-fault for SW/HW, not SW/SW or SW/SW/HW – e.g. precision.

System Architecture/Spec $F_1 \hat{} F_2 \hat{} F_3 \rightsquigarrow (S_1 \times S_2 \times S_3) \times S_4$

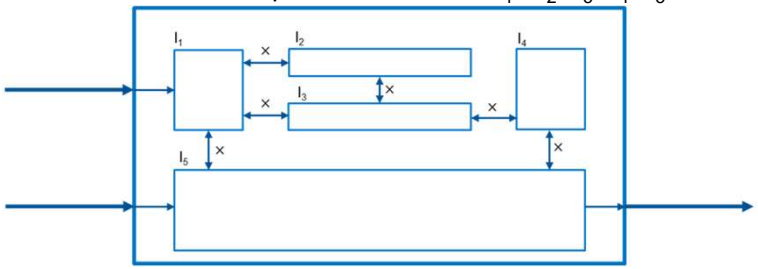


Functional Decomposition $F \rightsquigarrow (F_1 \hat{} F_2) \hat{} F_3$

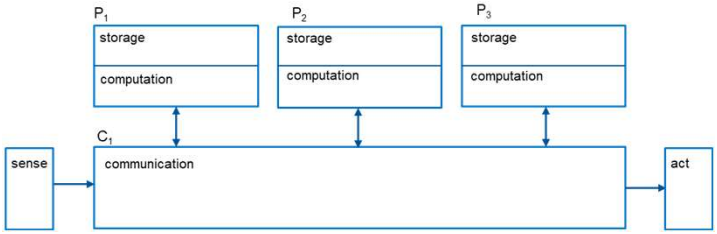


User Requirements F

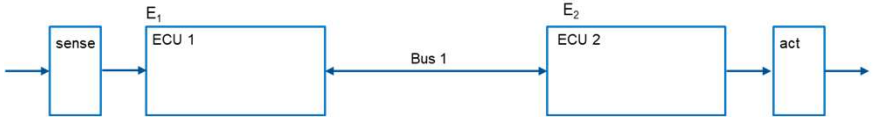
Software Implementation … $\rightsquigarrow I_1 \times I_2 \times I_3 \times I_4 \times I_5$



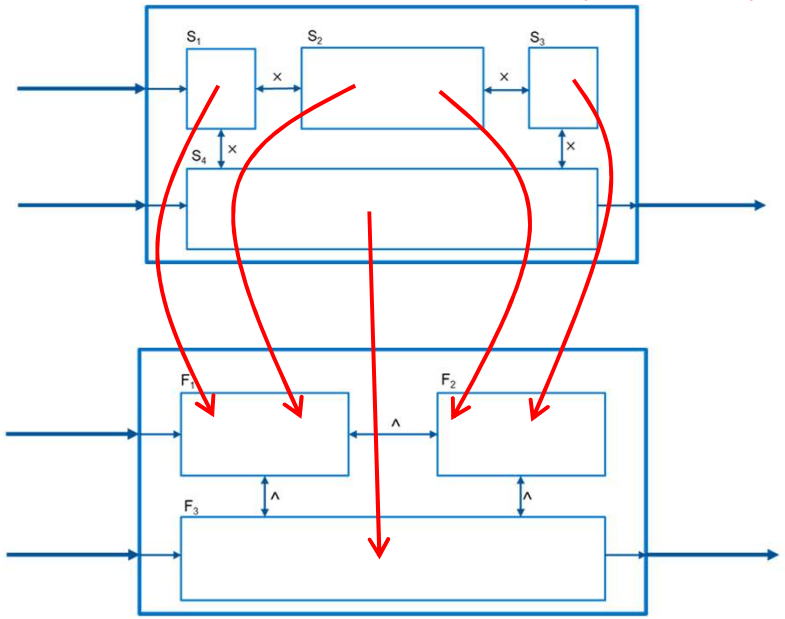
Abstract Hardware Design



Concrete Hardware



System **Architecture/Spec** $F_1 \hat{} F_2 \hat{} F_3 \rightsquigarrow (S_1 \times S_2 \times S_3) \times S_4$



High-level Architecture Verification:

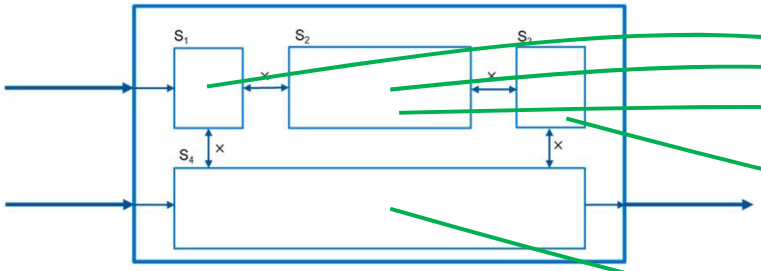
Check/assume that
- S1 and S2 correctly capture F1
- S2 and S3 correctly capture F2
- S4 correctly captures F3

But $(S_1 \times S_2 \times S_3)$ DO NOT correctly capture $F_1 \hat{} F_2$
or $(S_1 \times S_2 \times S_3) \times S_4$ DO NOT correctly capture $F_1 \hat{} F_2 \hat{} F_3$

These are integration faults that are (high-level) design faults!

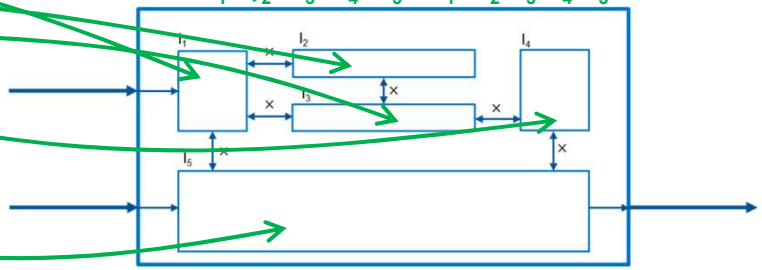
Reason: „Specs“ for subsystems/units are too liberal; design decisions for subsystems/units may be contradicting

System Architecture/Spec $F_1 \hat{} F_2 \hat{} F_3 \rightsquigarrow (S_1 \times S_2 \times S_3) \times S_4$

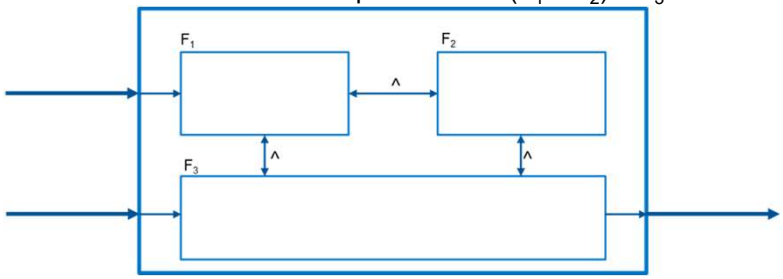


Component **Design and Implementation**

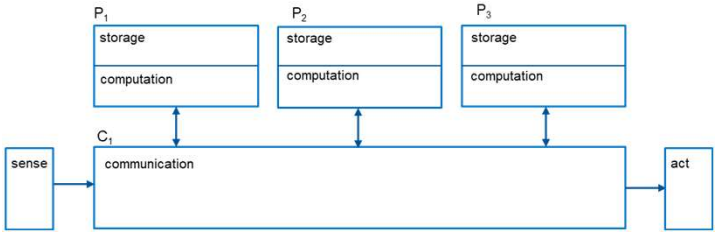
… $\rightsquigarrow J_1 \times J_2 \times J_3 \times J_4 \times J_5 \rightsquigarrow I_1 \times I_2 \times I_3 \times I_4 \times I_5$



Functional Decomposition $F \rightsquigarrow (F_1 \hat{} F_2) \hat{} F_3$

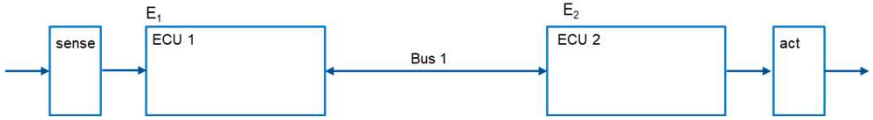


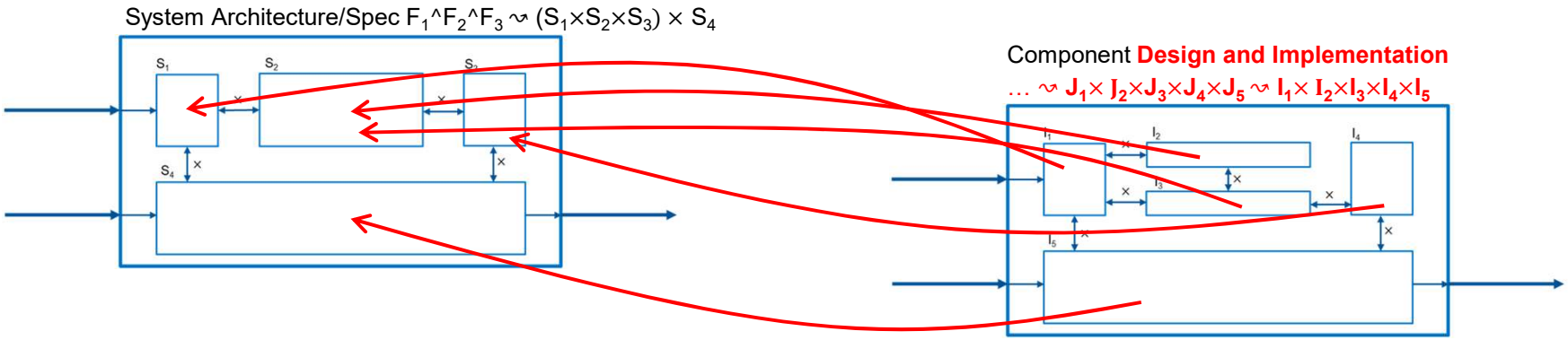
Abstract Hardware Design



User Requirements F

Concrete Hardware





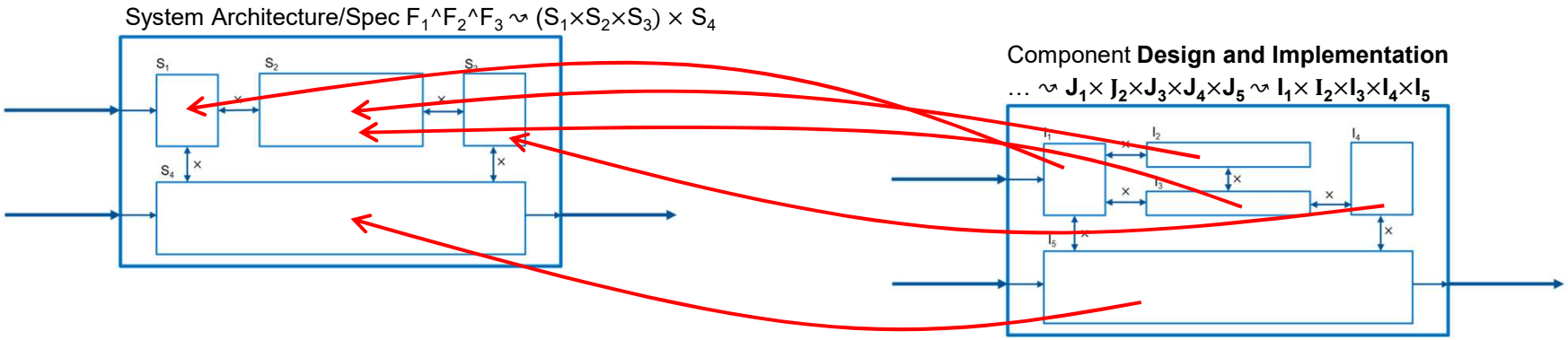
Low-Level Architecture Verification:

Assume that

- J1 correctly captures S1
- (J_2×J_3) correctly capture S2
- J4 correctly captures S3
- J5 correctly captures S4

But (J_1×J_2×J_3) DO NOT correctly capture (S_1×S_2), or (J_1×J_4) DO NOT correctly capture (S_1×S_4) and so on

These are integration faults that are (low-level) design faults!



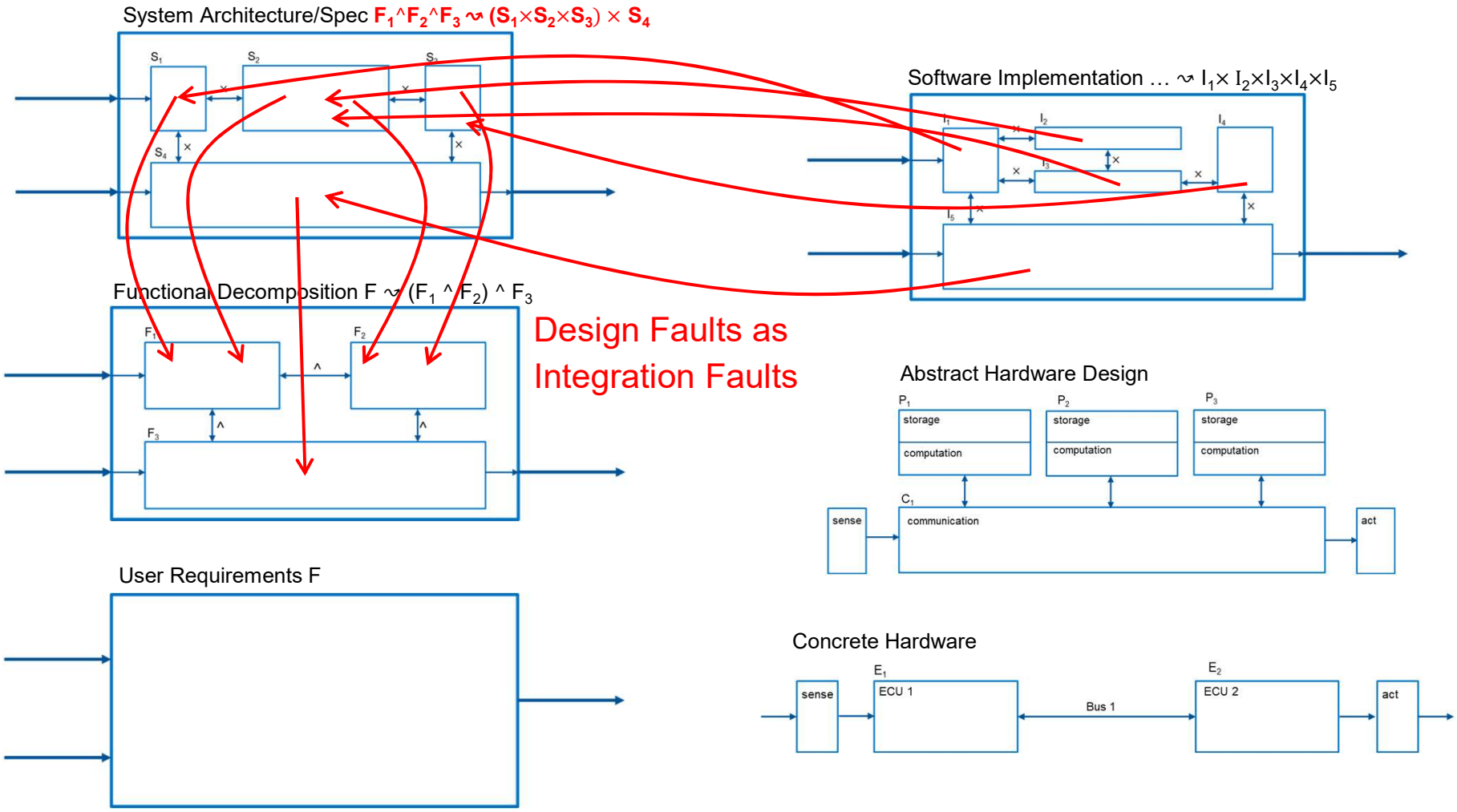
Assume that

- J1 correctly captures S1, **and I1 correctly implements J1**
- (J_2×J_3) correctly capture S2, **I2 correctly implements J2, I3 correctly implements J3**
- J4 correctly captures S3, and **I4 correctly implements J4**
- J5 correctly captures S4, and **I5 correctly implements J5**

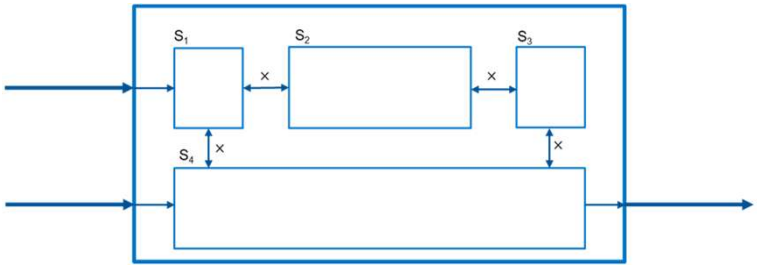
(J_1×J_2×J_3) correctly capture (S_1×S_2), **but (I_1×I_2×I_3) DO NOT correctly implement (J_1×J_2×J_3)**
Or (J_1×J_4) correctly capture (S_1×S_4), **but (I_1×I_4) DO NOT correctly implement (J_1×J_4)**

These are also low-level design faults!

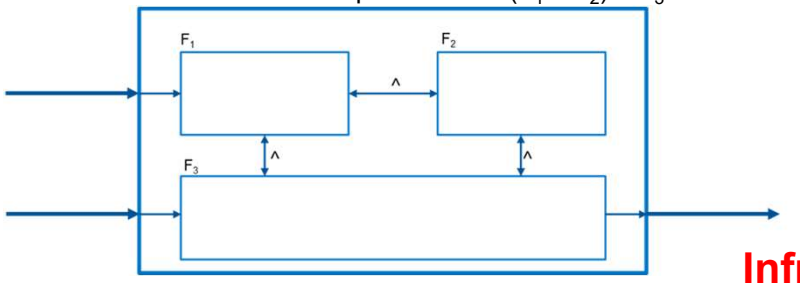
Assumption: Low-level design correct, unit implementations correct – **detectable only after implementation**



System Architecture/Spec $F_1 \hat{} F_2 \hat{} F_3 \rightsquigarrow (S_1 \times S_2 \times S_3) \times S_4$

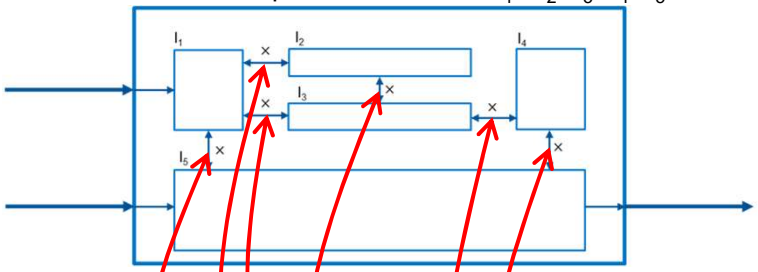


Functional Decomposition $F \rightsquigarrow (F_1 \hat{} F_2) \hat{} F_3$

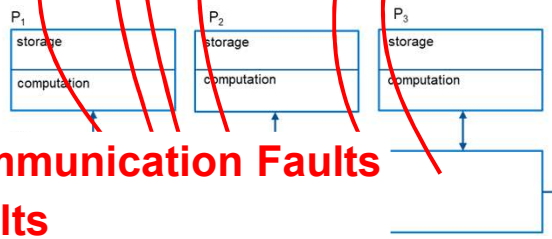


User Requirements F

Software Implementation … $\rightsquigarrow I_1 \times I_2 \times I_3 \times I_4 \times I_5$



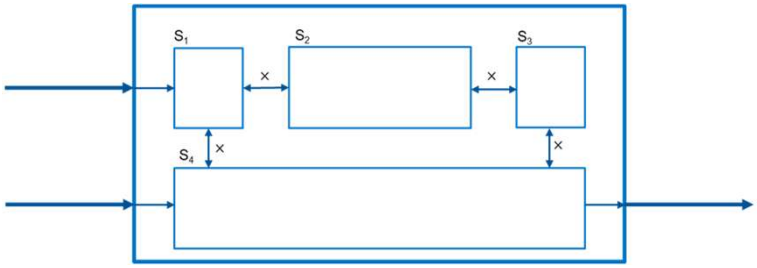
Abstract Hardware Design



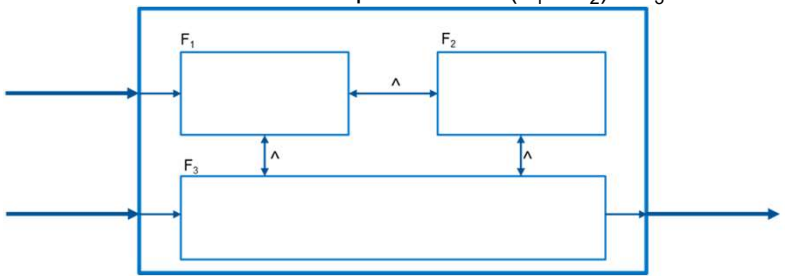
Infrastructure/Communication Faults as Integration Faults

Concrete Hardware

System/Architecture Spec $F_1 \hat{} F_2 \hat{} F_3 \rightsquigarrow (S_1 \times S_2 \times S_3) \times S_4$



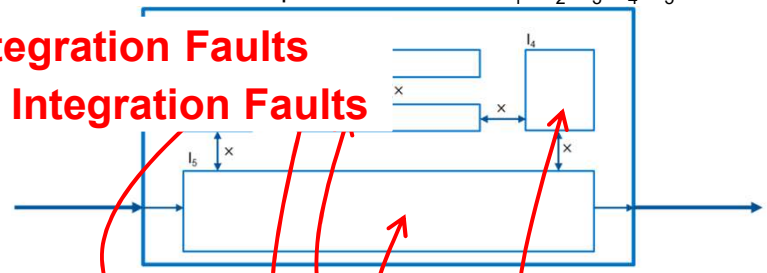
Functional Decomposition $F \rightsquigarrow (F_1 \hat{} F_2) \hat{} F_3$



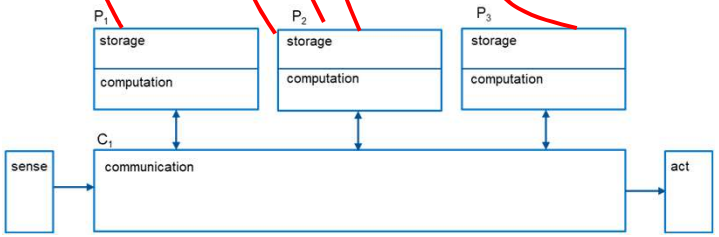
User Requirements F

HW/SW Integration Faults as Vertical Integration Faults

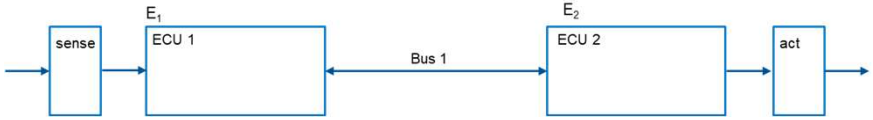
Software Implementation … $\rightsquigarrow I_1 \times I_2 \times I_3 \times I_4 \times I_5$



Abstract Hardware Design



Concrete Hardware



Integration Faults are Design or Infrastructure Faults

Design: Functionally decompose specification S into sub systems S$_1$, …, S$_n$,

$$S \rightsquigarrow S_1 + \cdots + Sn$$

such that (hopefully) $S_1 + \cdots + S_n \Rightarrow S$

Integration fault

Implementation: implement sub system specs S$_1$, …, S$_n$ by I$_1$, …, I$_n$,

$$S_i \rightsquigarrow I_i$$

such that (hopefully) $I_i \Rightarrow S_i$

Implementation fault

Unit Test: verify (and hopefully ensure) $I_i \Rightarrow S_i$

In theory: **integration** correct by-design because of transitivity – b… …y be faulty!

$$I_1 + \cdots + I_n \Rightarrow S$$

Integration fault

CLASS (a) — DESIGN FAULT, HORIZONTAL SOFTWARE INTEGRATION

Assumptions Not Established

(i) a

Sub-case 1 — contradictory

$[A_1]$ & $[A_2]$ inconsistent in itself. Unit 1 verified under “payload is JSON”, unit 2 under “payload is XML”. No architecture can establish both; deterministic, caught on the first end-to-end run.

Sub-case 2 — satisfiable

A and B are not inconsistent — nothing forbids them.
But they may be incompatible: I_2 emits values with property P_2, I_1 was verified under input assumption P_1, and P_2 does not imply P_1.

A single input from P_2 outside P_1 suffices — so the system survives for months, until the first apostrophe in a surname or the first 29 February. The fault was there on day one; only the witness was missing.

Instances for P: units (ground software in lbf·s, navigation in N·s, both modules correct) · frequency · byte order · timing · assumptions about call history · et cetera

Question

If the platform keeps exactly to W — does the fault go away?
No ⇒ (a)

Attribution

None. Three equally admissible sites:
loosen I_1, tighten I_2, or insert an adapter.

CLASS (b) — INFRASTRUCTURE FAULT, VERTICAL INTEGRATION WITH COMM INFRASTRUCTURE

The Platform Does Not Satisfy the Assumption

(ii) b

NOT([W_real] → [W])

The design step was correct under W. But W describes reality wrongly.

- Synchronous where asynchronous was assumed
- Latency above the stated bound; lossy channel
- Clock drift; message duplication; reordering

Two repairs, one of which is a reclassification

Fix the platform — configuration or replacement.

Or correct W to the truth. Then the same fault reappears as a now-visible design fault and is handled as (a).

Question

Does the fault vanish if the platform satisfies W exactly? Yes ⇒ (b)

Attribution

None — it sits beneath all units, in no component at all.

Example: W says FIFO per channel. The deployed middleware uses multipath routing and delivers out of order.

CLASS (b) — EXAMPLE

Billed Twice

a

I_1 = charge(order) implicit A: "each order-ID reaches me at most once"

1. **The unit test calls charge once**

   A is trivially satisfied inside the harness, so nobody ever writes it down.

2. **I_2 sits on an *at-least-once* channel**

   It retries on timeout. That is what W didn't say.

3. **The customer is billed twice**

   Both components satisfy their specifications. The middleware behaves exactly as described but too abstractly.

The architecture attached a component requiring exactly-once to a channel that does not provide it. Repair: dedup key in the protocol, or make I_1 idempotent, or add an adapter — an architectural decision, not a bug fix.

CLASS (c) — A UNIT FAULT, NOT AN INTEGRATION FAULT – VERTICAL INTEGRATION WITH HARDWARE

The Unit Fails Its Own Spec

(iii) C

$$\text{NOT}(\ [I_1] \rightarrow [J_1]\ )$$
$$\text{or} \quad \text{NOT}(\ [I_2] \rightarrow [J_2]\ )$$

Either an ordinary bug — or an over-idealized spec no implementation on this machine can satisfy: [J] computes in $\mathbb{R}$ where the machine has IEEE-754; [J] assumes unbounded memory.

Why it surfaces at integration

The offending input $x^* \in X \setminus X\_test$ only becomes reachable there. The partner component or the platform's timing produces it; the harness never did.

Attribution — settled

Exactly one component violates its own specification in a given W_real. The artifacts single it out, and nothing else has to move. That is the dividing line.

Question: Is there a run in which a component violates its own specification?

Difference with (b): Infrastructure assumption problem that surfaces for *one* unit

Vertical Integration with Hardwre

Test and Proof — What Each One Buys

Test

- Samples X and extrapolates. Misses x* until something makes it reachable.
- Scales to whole systems and to the real platform — the only method that observes W_real at all.
- Needs an oracle only for the cases it runs, so [J] can stay informal.
- A_1 and A_2 live in the harness, unstated.
- Cannot pass vacuously: a contradictory setup runs no cases.

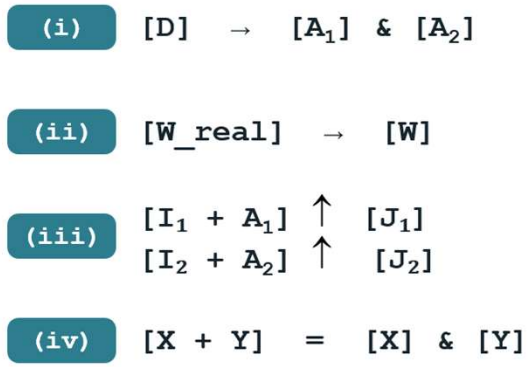
Proof

- Quantifies over entire input range. No extrapolation.
- Scales in the input space, not in system size — cost grows with component complexity, and whole-system proofs stay rare.
- Requires [J] and [W] and [A] stated precisely. Often the real work, and often the real obstacle.
- Forces part of A and B into the open — but only the part inside the model. Scheduler, clock, network, floating point and memory usually sit outside it.
- Can pass vacuously.

Which obligation each one reaches

A proof is the strong instrument for (iii) across all inputs, and the only instrument for (iv). A test is the only access to (ii), because (ii) is a claim about hardware nobody can prove from a model of the code. Proof settles (i) if A_i written down somewhere. (iv) doesn't matter for tests.

Abstraction behind A may miss faults.



THE CRITERION

What Makes an Integration Fault One

A unit fault

Attributable. There is exactly one component whose implementation violates its own specification. The artifacts themselves single it out, and the diagnosis is finished when they do.

An integration fault

Not attributable. Every component satisfies its own specification, so no single component can be held responsible. The fault lies in the relation between the parts, which the design step asserted.

A fault is an integration fault if it cannot be attributed to any single component: each part meets its own specification, and the specifications do not determine where to intervene.

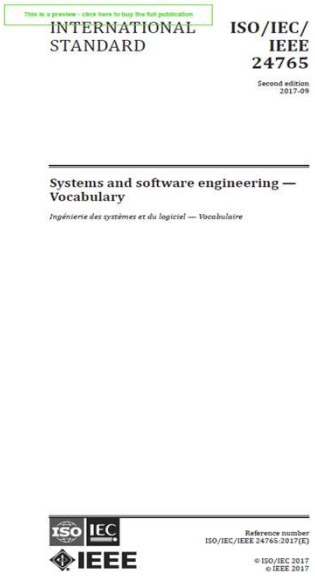
This does not say the repair must touch several components: it usually can be done in any one component. If two specifications are silent about the wire format and the implementations disagree, either side can be changed, and no specification needs to move. That is precisely the point: the specifications leave the choice open, so choosing is a design decision and not a diagnosis.

Test: is there a component whose implementation violates its own specification? If yes, attribution is settled. If no, the repair site is a decision someone has to make and record.

Confusion Resolved

Integration Test [ISO/IEC/IEEE 24765:2017(E)]:
"progressive **linking and testing of programs or modules** in order to ensure their **proper functioning** in the complete system"
→ **testing for design faults**:
conflicting assumptions — every part meets its own spec, yet the composition fails

Integration Testing [ISO/IEC/IEEE 24765:2017(E)]:
"**testing** in which software **components**, hardware components, or both are **combined** and tested to **evaluate the interaction among them**"
→ **testing for communication/infrastructure faults**:
the connection itself (middleware, bus, timing) doesn't behave as the design assumed



From this perspective, testing for „JSON vs. XML“ or „mps vs. kph“ is a design fault, not a communication fault.

Bottom line

Integration problems may occur …

- because of a **design fault** that is understood only after implementation
- because of deployment problems when **hardware doesn‘t meet assumptions of software**
- because of deployment problems when **communication doesn‘t meet assumptions of software**

Assumptions that lead to problems are implicit and not recorded in specifications!

Good Tests

Test: Input plus expected output (plus infrastructure conditions) → require a specification!
~~„reveal faults“~~
~~„reveal *potential* faults“~~
„reveal potential bugs with good cost effectiveness (low risk)“

Integrated components: against which specification?

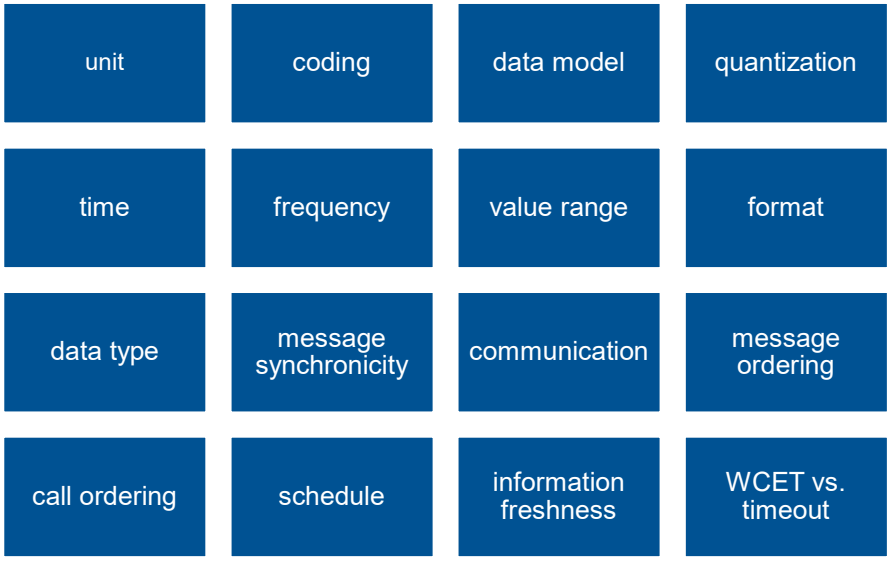
Which potential faults? Integration tests address potential integration faults!

First idea: **Potential faults as implicit specification** – „X must not be the case“

Integration faults

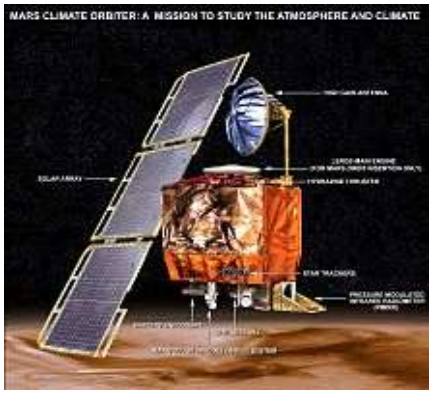
If they exist, subsystem specs are **abstractions** and therefore (necessarily) incomplete

Aspects that may not have been specified include



Integration Faults: Unit, Mars Climate Orbiter

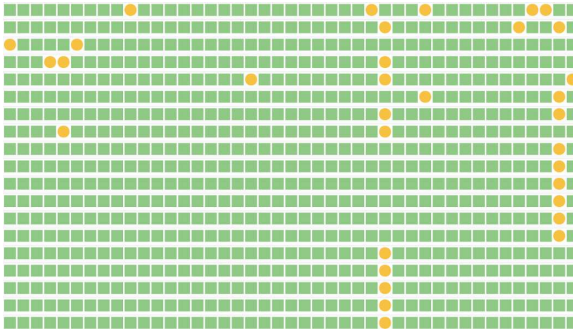
Subsystem specs are **abstractions** and therefore (necessarily) incomplete



unit	coding	data model	quantization
time	frequency	value range	format
data type	message synchronicity	communication	message ordering
call ordering	schedule	information freshness	WCET vs. timeout

Integration Faults: WCET, Flaky Tests

Subsystem specs are **abstractions** and therefore (necessarily) incomplete



unit	coding	data model	quantization
time	frequency	value range	format
data type	message synchronicity	communication	message ordering
call ordering	schedule	information freshness	**WCET vs. timeout**

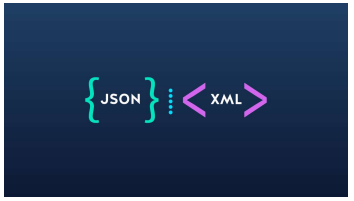
Problem? Specifications Partial and Inconsistent!

Because a fault is usually located in one unit, we could, *in principle*, reveal these integration faults with unit tests – if we had sufficiently precise unit specifications.

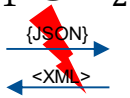
For this reason alone, the boundary between unit and integration tests is blurred!

For some failures, there is no unique unit to blame …

… because unit specs are partial and refinements may be inconsistent!



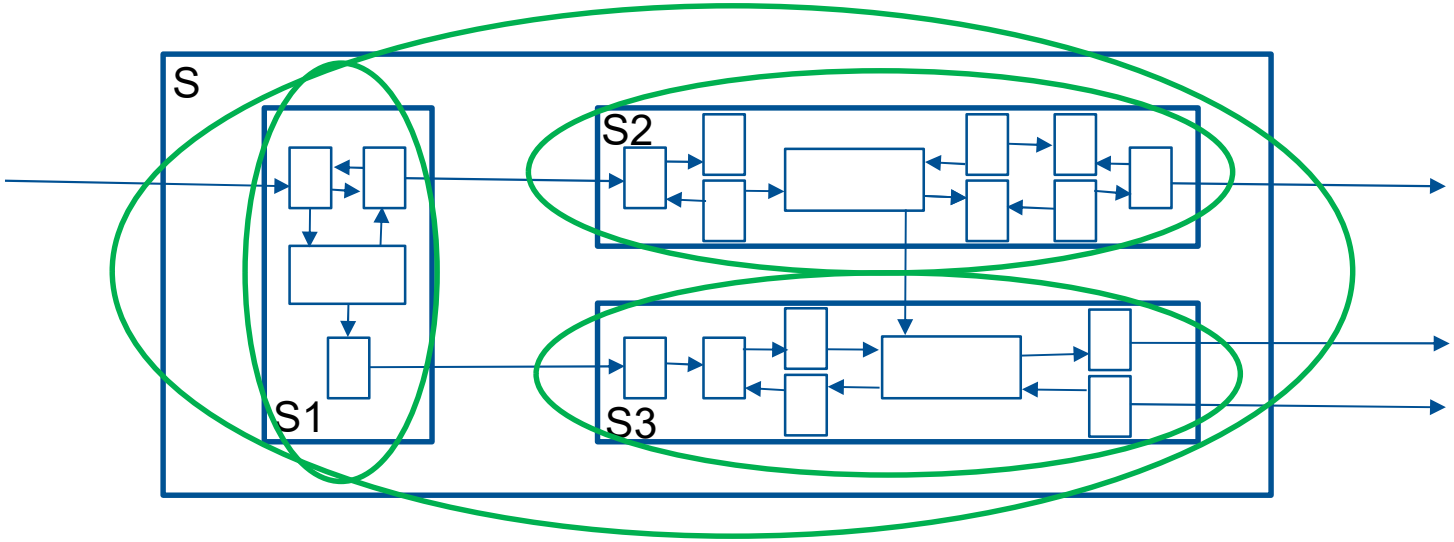
$$S \rightsquigarrow S_1 \oplus S_2 \ldots \oplus Sn$$



Specs for SW Subsystems?

Subsystem and system vs. integration tests:

Possibly specs for subsystems, (what if we don‘t know what constitutes the subsystem?)



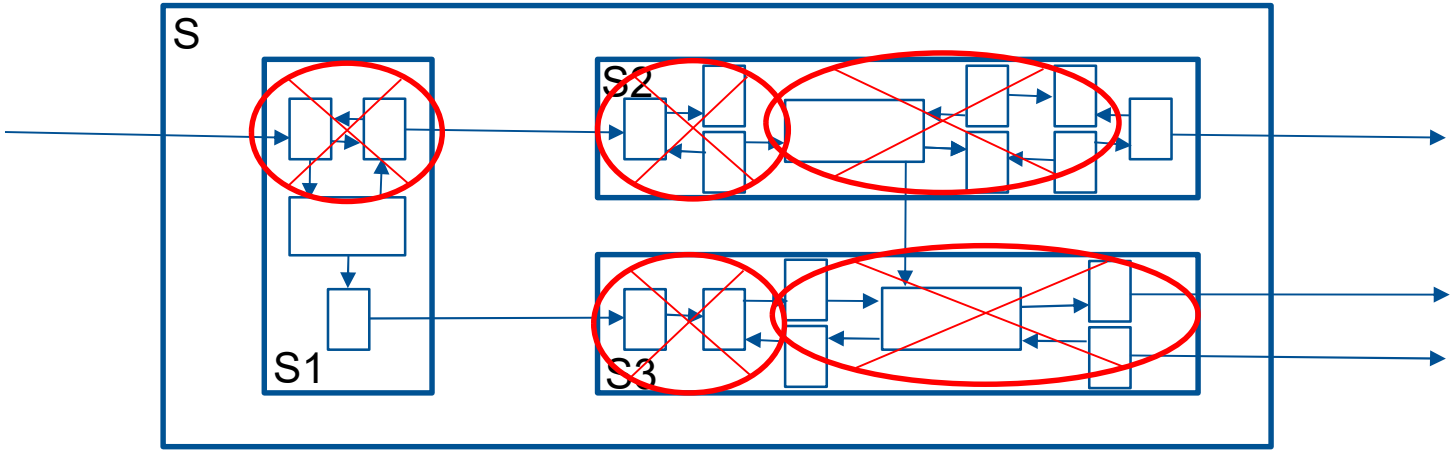
Specs for Arbitrary SW Compositions?

Subsystem- and system- vs. integration tests:

Possibly specs for subsystems, usually no spec for the composition of arbitrary sets of units!

→ explorative matching of interfaces instead

(what if we don‘t know what constitutes the subsystem?)



Not all System Failures detectable with Unit/Integration Tests: Systemic („Emergent“) Properties

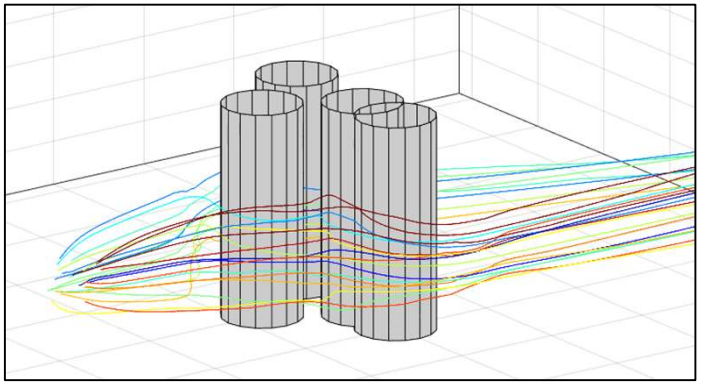
Performance

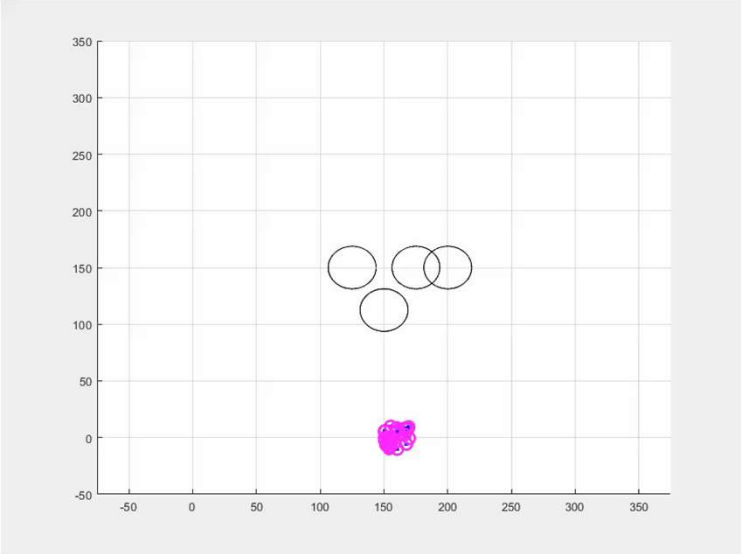
Security: weakest link, confidentiality not compositional

Functionality:

minimum distance to neighboring drones or obstacles

$$S_1 \oplus \cdots \oplus S_n \Rightarrow S$$





E. Soria, F. Schiano, and D. Floreano, “SwarmLab: a Matlab Drone Swarm Simulator,” *IEEE/RSJ Intl. Conf. on Intelligent Robots and Systems (IROS), 2020*

A Matter of Perspective: the „System“

https://www.youtube.com/watch?v=2mBjYZTeaTc

Unit-level properties of the ACC can be optimized – when considering strings of vehicles as the system

$S_1 \oplus \cdots \oplus S_n \Rightarrow S$

ACC planner parameters speed, acceleration, time distance

traffic string stability – possibly not considered

https://stern.cege.umn.edu/research

A design fault, not an integration fault. Can be optimized using a (system-level) simulation of the cars in front.

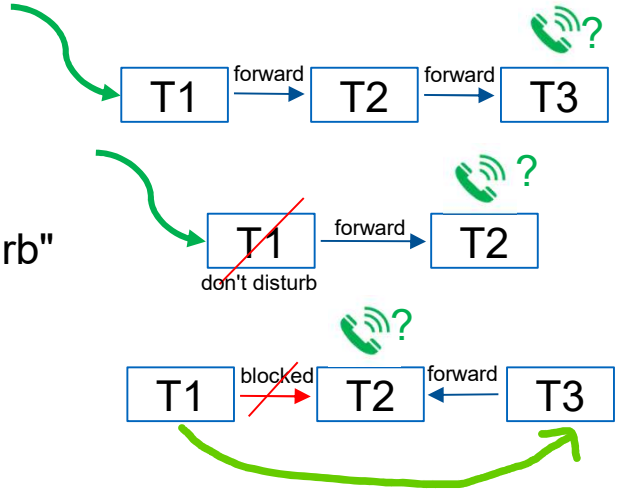
[Hauer, Stern, Pretschner: Selecting Flow-Optimal System Parameters for Automated Driving Systems, ITSC 2019]

… Feature Interactions …

Phone number T1 forwarded to T2 forwarded to T3:
Is a call to T1 forwarded to T3?

T1 "do not disturb", T1 forwarded to T2, T2 no "do not disturb"
Is a call to T1 forwarded to T2?

T1 blocks outgoing calls to T2. T3 forwarded to T2.
Is a call from T1 to T3 forwarded to T2?



Another design fault that is not an integration fault.
Underlying problem similar: incomplete requirements for the overall system

Now what?

Testing is risk-based hence defect-based

Integration testing targets integration faults: design and infrastructure faults

Underlying problem: specs, abstractions, assumptions.

Testing the right remedy? No. But the only realistic one, I am afraid.

Use information about integration faults to derive and assess integration test suites; use LLMs to generate tests and specs for subsystems.

Integration Faults as Design Faults in Disguise

Integration testing usually is a specification problem.

1. **Integration is based on a shared understanding of the world**

   Explicit contracts (APIs, data models) plus implicit assumptions: timing, units, ranges, ordering.

2. **Integration faults surface only under composition**

   Every part meets its own spec, yet the system fails — the fault was in the design or infrastructure.

3. **In principle, the unit / integration boundary is artificial**

   With sufficiently complete specs we'd catch them earlier. The real gap is the missing specification.

4. **Remedy 1: Test against negative specifications**

   No spec? Catalog the specific faults composition can produce — the systemic ones it can't reach.

5. **Remedy 2: Surface the implicit background knowledge**

   Generative AI and compound engineering can help fill in the unwritten assumptions.